

מבוא למדעי המחשב

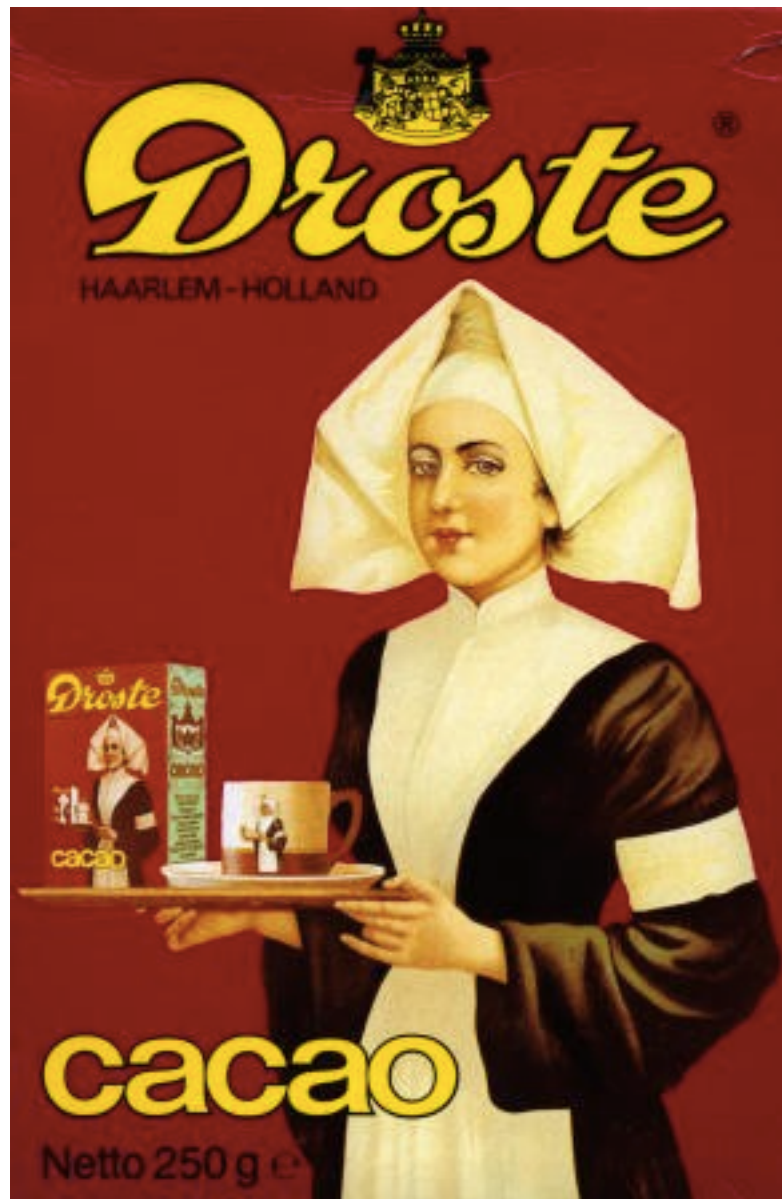
שיעור 5: רקורסיה, מבני נתונים

לקריאה נוספת:

- Sarina Singh et al., *India travel guide* 15th ed., Lonely Planet, 2013.
- Franz Kafka, *Die Verwandlung*, Kurt Wolff Verlag, Leipzig, 1915.
- Tirumalāmbā, *Varadāmbikā Pariṇaya Campū*, c. 1540.

רקורסיה

רקורסיה



- הגדרה מילולית (מוויקיפדיה):
תופעה שכל מופע שלה מכיל מופע
נוסף שלה, כך שהיא מתרחשת
ומשתקפת בתוך עצמה שוב ושוב.
- דוגמה: רקורסיה וויזואלית כמו זו ←
שנקראת אפקט דרוסטה.
- ברקורסיה **סופית** (או עוצרת), מעבר
לרמה מסויימת אין רקורסיה.
- ברקורסיה **אינסופית**, כל רמה מכילה
רמות משנה מאותו סוג.

רקורסיה בשירה



קטע מ-
יהודה עמיחי, שלוחה גדולה

רקורסיה בשירה

The vermin only teaze and pinch
Their foes superior by an inch.
So, naturalists observe, a flea
Has smaller fleas that on him prey;
And these have smaller still to bite 'em,
And so proceed ad infinitum.

...

קטע מ-

Jonathan Swift, On Poetry: A Rhapsody

תכנות רקורסיבי

- פונקציה שהמימוש שלה מכיל קריאה לעצמה נקראת **פונקציה רקורסיבית**.
- קריאה לפונקציה מתוך עצמה נקראת **קריאה רקורסיבית**.
- על מנת שהביצוע לא ימשיך לעד, חשוב להקפיד על:
 1. הגדרה ומימוש של **תנאי עצירה**; כאשר התנאי מתקיים, אין קריאה רקורסיבית.
 2. בכל קריאה רקורסיבית מתקדמים לעבר קיום תנאי העצירה.

רקורסיה ואינדוקציה

- יש הרבה סדרות מתמטיות מעניינות שמוגדרות באינדוקציה. כלומר, ערך האבר ה- n -י מוגדר על סמך ערכים קודמים בסדרה.

- דוגמה: הסדרה $1!, 2!, 3!, \dots$ מוגדרת כך:

$$1! = 1$$

$$n! = n \cdot (n-1)!$$

- בסדרה כזו קל לכתוב קוד שמחשב את האבר ה- n -י ברקורסיה.

חישוב $n!$

Python 3.4.1: factorial.py - /Users/yrabani/Documents/5 פייתון/תשעה/שיעור - Code/factorial.py

```
def factorial(n):  
    """  
    This recursive function computes n factorial (n!).  
  
    input: n >= 1 and type(n) == int  
    output: a positive int that equals n!  
    errors and exceptions: none  
  
    performance guarantees: O(n) time and space  
    """  
  
    if n == 1:    # the bottom of the recurrence  
        return 1  
    else:        # n > 1  
        return n*factorial(n-1)
```

חישוב $n!$

Python 3.4.1: factorial.py - /Users/yrabani/Documents/5 פייתון/תשעה/שיעור - Code/factorial.py

```
def factorial(n):  
    """  
    This recursive function computes n factorial (n!).  
  
    input: n >= 1 and type(n) == int  
    output: a positive int that equals n!  
    errors and exceptions: none  
  
    performance guarantees: O(n) time and space  
    """  
  
    if n == 1:    # the bottom of the recurrence  
        return 1  
    else:        # n > 1  
        return n*factorial(n-1)
```

שימוש לדוגמה:

```
f = factorial(5)  
print(f)
```

מה יודפס?

Ln: 7 Col: 31

חישוב $n!$

Python 3.4.1: factorial.py - /Users/yrabani/Documents/5 פייתון/תשעה/שיעור - Code/factorial.py

```
def factorial(n):  
    """  
    This recursive function computes n factorial (n!).  
  
    input: n >= 1 and type(n) == int  
    output: a positive int that equals n!  
    errors and exceptions: none  
  
    performance guarantees: O(n) time and space  
    """  
  
    if n == 1:    # the bottom of the recurrence  
        return 1  
    else:        # n > 1  
        return n*factorial(n-1)
```

שימוש לדוגמה:

f = factorial(5)

print(f)

120 מה יודפס?

Ln: 7 Col: 31

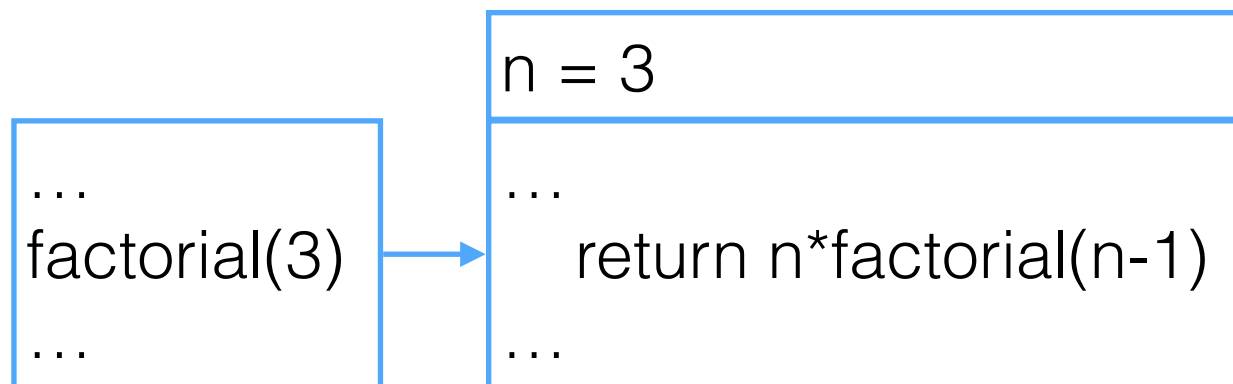
ניהול הזיכרון

- בכל קריאה לפונקציה נוצר עותק חדש שלה. בעותק:
 - הקצאה חדשה של כל המשתנים הפנימיים, כולל הפרמטרים שמופיעים בחתימה שלה.
 - השמה התחלתית של ערכים לפרמטרים לפי מה שהועבר בקריאה אליה.
 - איתחול של הפקודה הבאה לבצע בקוד - בעותק החדש זו הפקודה הראשונה בפונקציה.

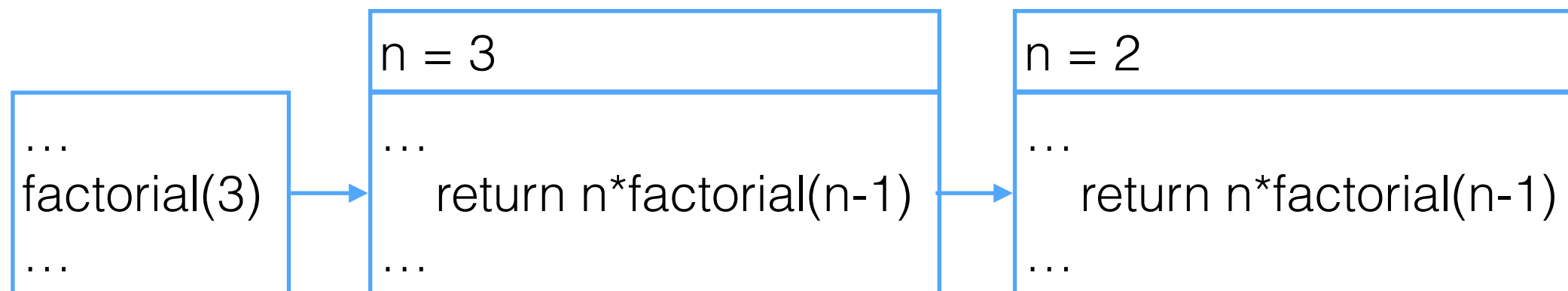
איך זה נראה?

```
...  
factorial(3)  
...
```

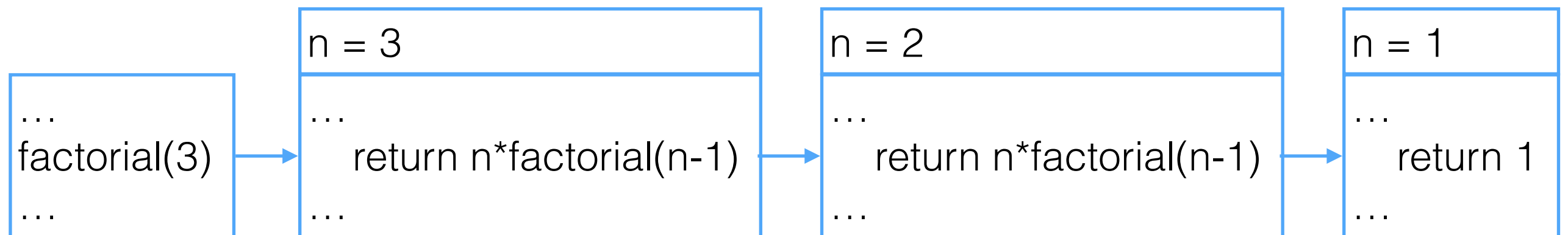
איך זה נראה?



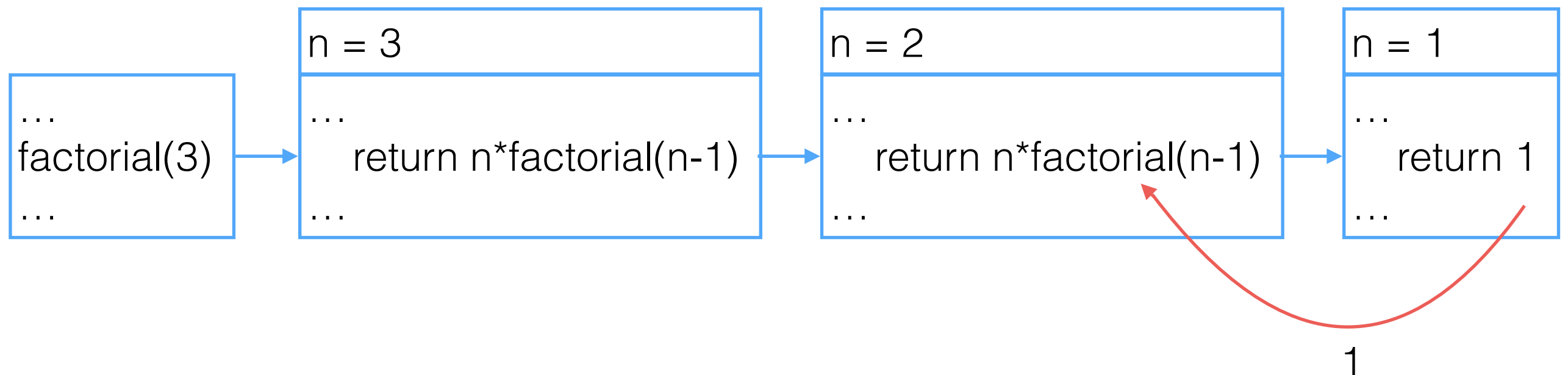
איך זה נראה?



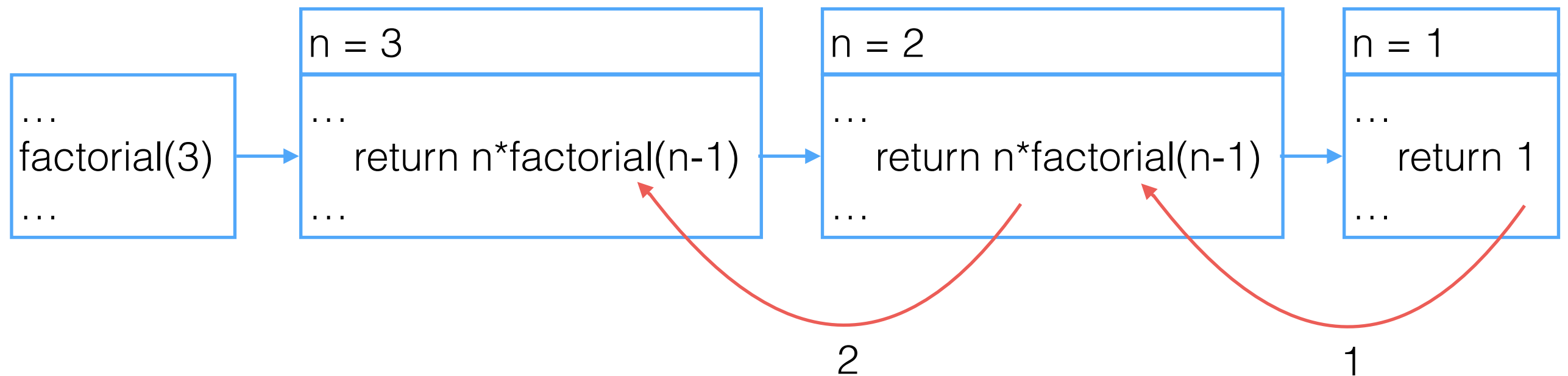
איך זה נראה?



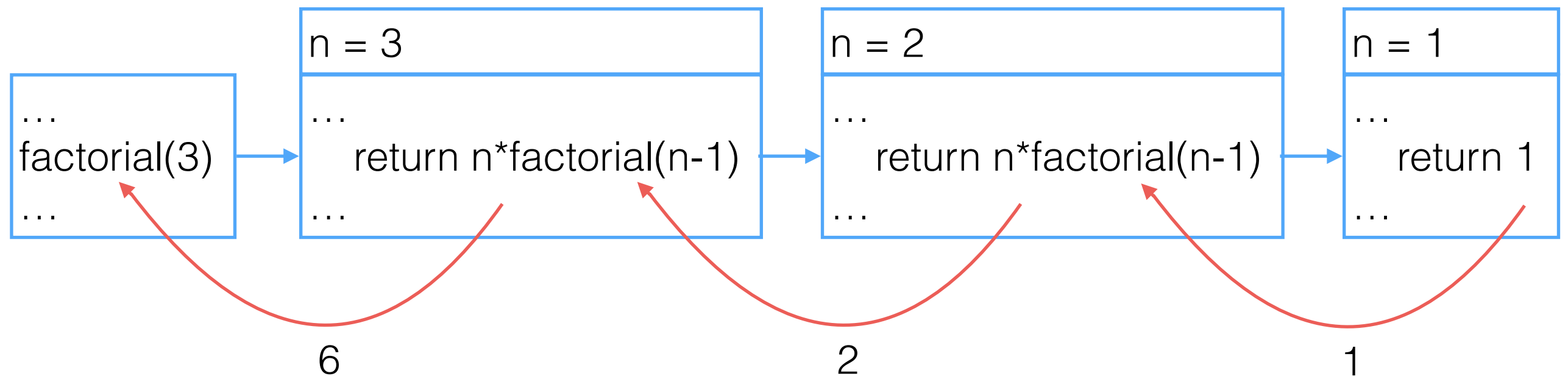
איך זה נראה?



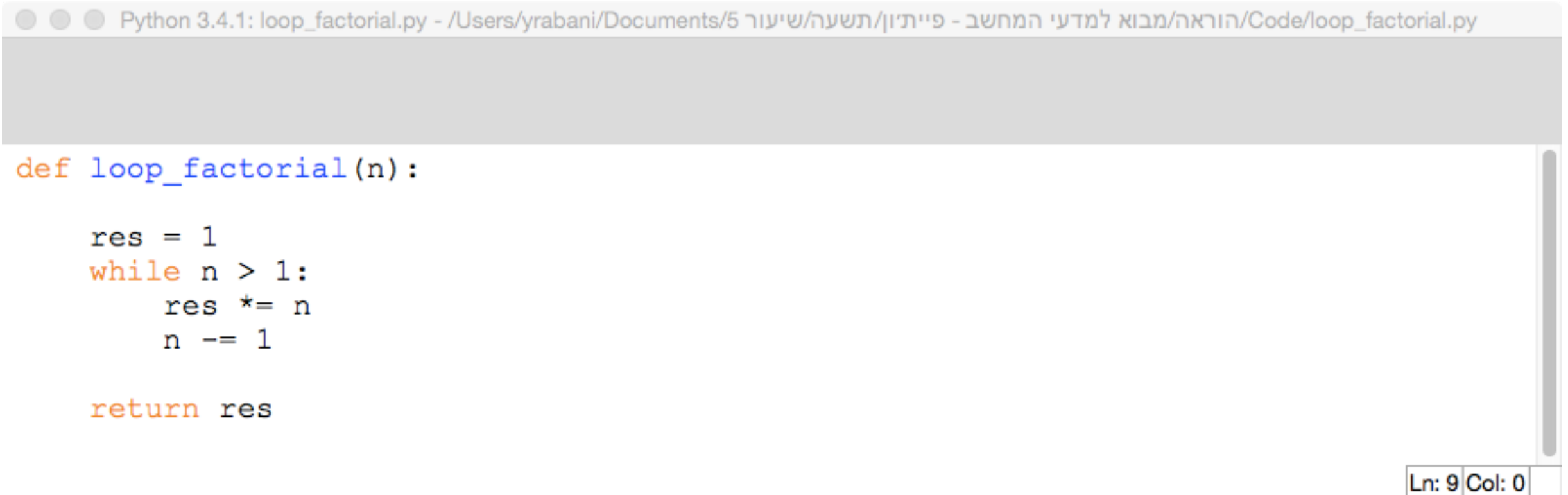
איך זה נראה?



איך זה נראה?



להשוואה

A screenshot of a Python IDE window. The title bar shows the file path: Python 3.4.1: loop_factorial.py - /Users/yrabani/Documents/5 פייתון/תשעה/שיעור 5/Code/loop_factorial.py. The main area contains a Python function definition for a loop-based factorial. The code is as follows:

```
def loop_factorial(n):  
    res = 1  
    while n > 1:  
        res *= n  
        n -= 1  
  
    return res
```

The status bar at the bottom right indicates the cursor is at line 9, column 0.

הקוד הרקורסיבי תואם להגדרה באינדוקציה ויפה יותר
אבל דורש יותר זיכרון במהלך הביצוע - בתחתית הרקורסיה
מוחזקים בו-זמנית n עותקים של `factorial`.

דוגמה: סדרת פיבונצ'י

Python 3.4.1: fibonacci.py - /Users/yrabani/Documents/5 פייתון/תשעה/שיעור 5 - פייטון/הוראה/מבוא למדעי המחשב - Code/fibonacci.py

```
def fibonacci(n):  
    """  
    This function computes the n-th Fibonacci number recursively.  
    The n-th Fibonacci number F(n) is defined inductively as follows:  
    F(1) = F(2) = 1  
    F(n) = F(n-2) + F(n-1) for all n > 2.  
  
    input: type(n) == int and n > 0  
    output: an int F(n), the n-th number in the sequence 1, 1, 2, 3, 5, 8, ...  
    errors and exceptions: none  
  
    performance guarantees: time 2**O(n), space O(n)  
    """  
  
    if n > 2: # we assume that the recursive calls return the correct value  
        return fibonacci(n-2) + fibonacci(n-1)  
    else: # the base cases n==1 or n==2  
        return 1
```

הערכת ביצועים

- נספור כמה קריאות מתבצעות ל-fibonacci.
- נסמן ב- $T(n)$ את מספר הקריאות שנדרשות כדי לחשב את $\text{fibonacci}(n)$.

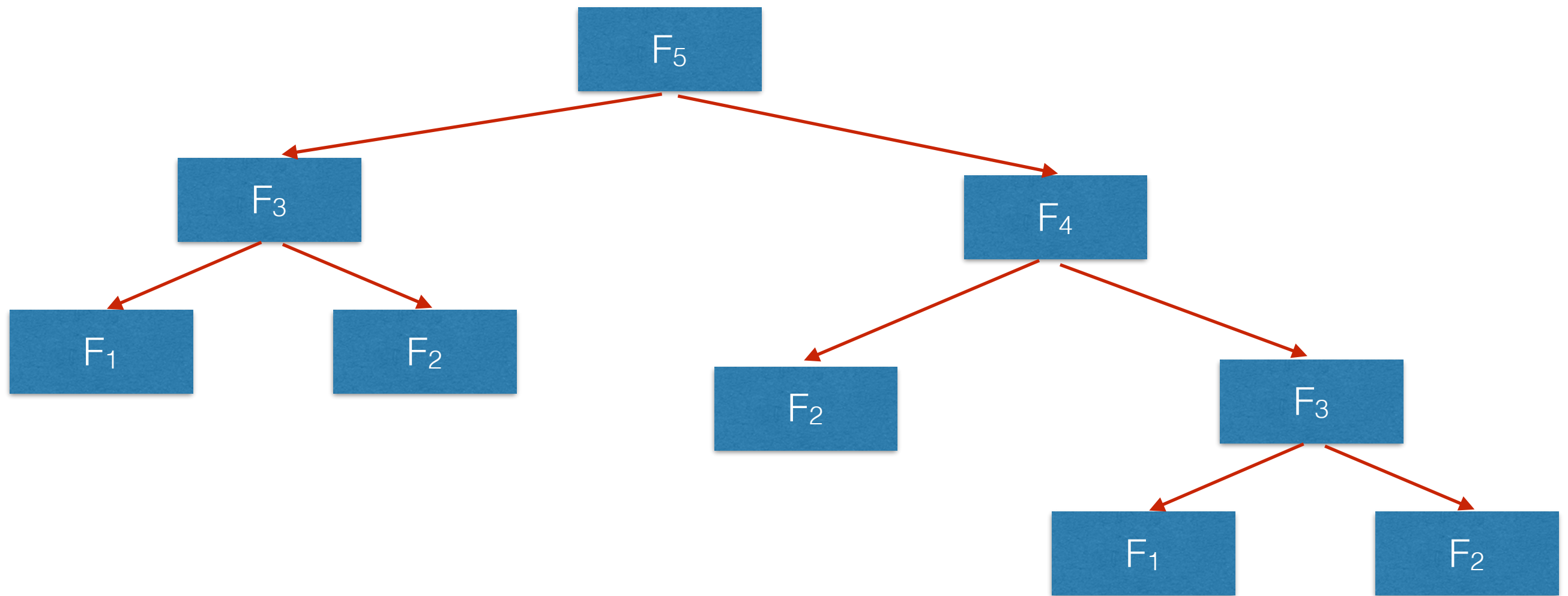
$$T(1) = T(2) = 1$$

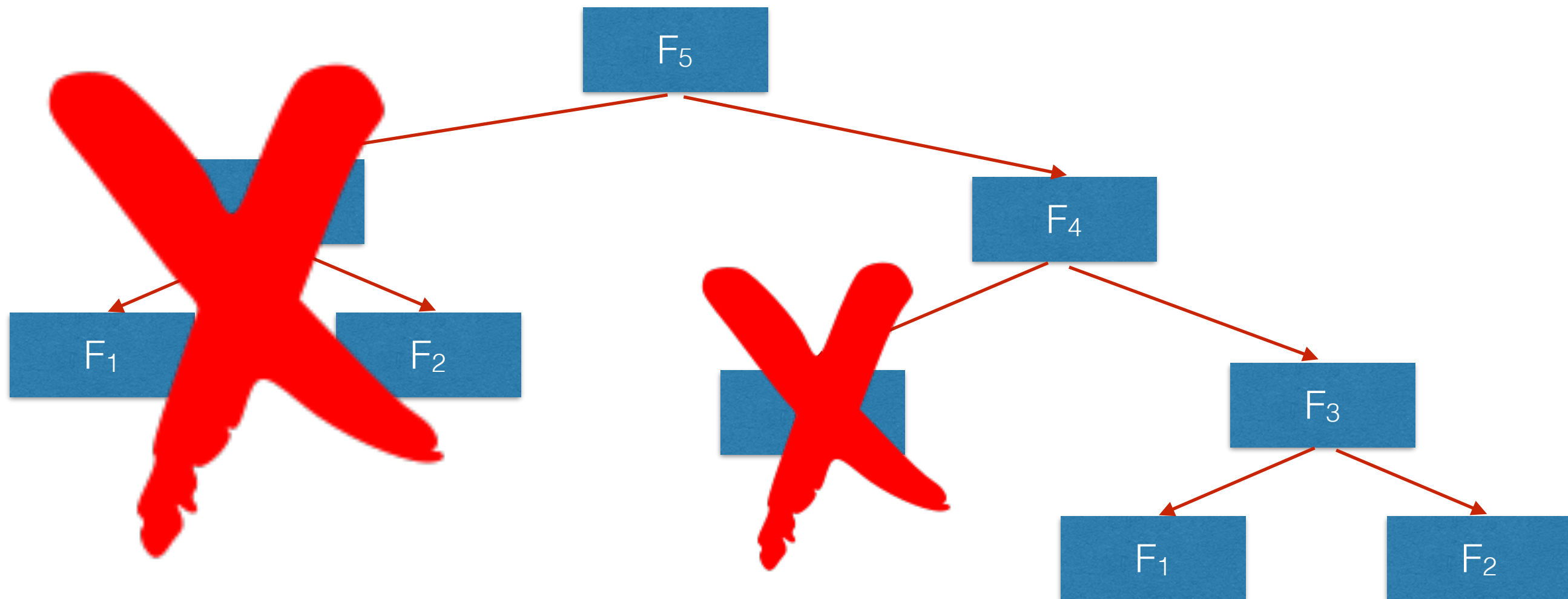
$$T(n) = 1 + T(n-2) + T(n-1)$$

- הפיתרון מקיים $T(n) = 2F_n - 1$ (הוכחה באינדוקציה).
- ידוע כי F_n הוא השלם הקרוב ביותר ל- $\Phi^n / \sqrt{5}$ ($\Phi \approx 1.618\dots$).
- זה מספר שגדל בקצב מאוד מהיר! למשל: $F_{100} \approx 3.54 \times 10^{20}$.
- צריכת הזיכרון: בשלב מסויים בריצה פתוחים בבת אחת n עותקים של הפונקציה.

מה הבעייה?

- אנחנו מחשבים מחדש אותם ערכים פעם אחר פעם.
- למשל, עבור $n=10$, הקריאה $\text{fibonacci}(n-2)$ מחשבת את F_8 וגם הקריאה $\text{fibonacci}(n-1)$ מחשבת את F_8 באחת הקריאות הרקורסיביות שם.
- הערכים F_1 ו- F_2 מוחזרים סך הכל F_n פעמים.
- אם נזכור את שני הערכים האחרונים, לא נצטרך לחשב מחדש ברקורסיה את כל הערכים הקודמים.





מימוש בלולאה

Python 3.4.1: loop_fibonacci.py - /Users/yrabani/Documents/5 פייתון/תשעה/שיעור - פיתרון/Code/loop_fibonacci.py

```
def loop_fibonacci(n):  
    """  
    This function computes the n-th Fibonacci number in a loop.  
  
    input: type(n) == int and n > 0  
    output: an int F(n), the n-th number in the sequence 1, 1, 2, 3, 5, 8, ...  
    errors and exceptions: none  
  
    performance guarantees: time O(n), space O(1)  
    """  
  
    if n > 2:  
        prev = 1           # F(1)  
        curr = 1           # F(2)  
        for i in range(3, n+1): # compute F(3), F(4), F(5), ..., F(n)  
            new = prev + curr # compute the next entry in the sequence  
            prev = curr  
            curr = new  
  
        return curr  
    else:  
        return 1
```

מימוש בלולאה

Python 3.4.1: loop_fibonacci.py - /Users/yrabani/Documents/5 פייתון/תשעה/שיעור 5 - פיתרון/Code/loop_fibonacci.py

```
def loop_fibonacci(n):  
    """  
    This function computes the n-th Fibonacci number in a loop.  
  
    input: type(n) == int and n > 0  
    output: an int F(n), the n-th number in the sequence 1, 1, 2, 3, 5, 8, ...  
    errors and exceptions: none  
  
    performance guarantees: time O(n), space O(1)  
    """  
  
    if n > 2:  
        prev = 1           # F(1)  
        curr = 1           # F(2)  
        for i in range(3, n+1): # compute F(3), F(4), F(5), ..., F(n)  
            new = prev + curr # compute the next entry in the sequence  
            prev = curr  
            curr = new  
  
        return curr  
    else:  
        return 1
```

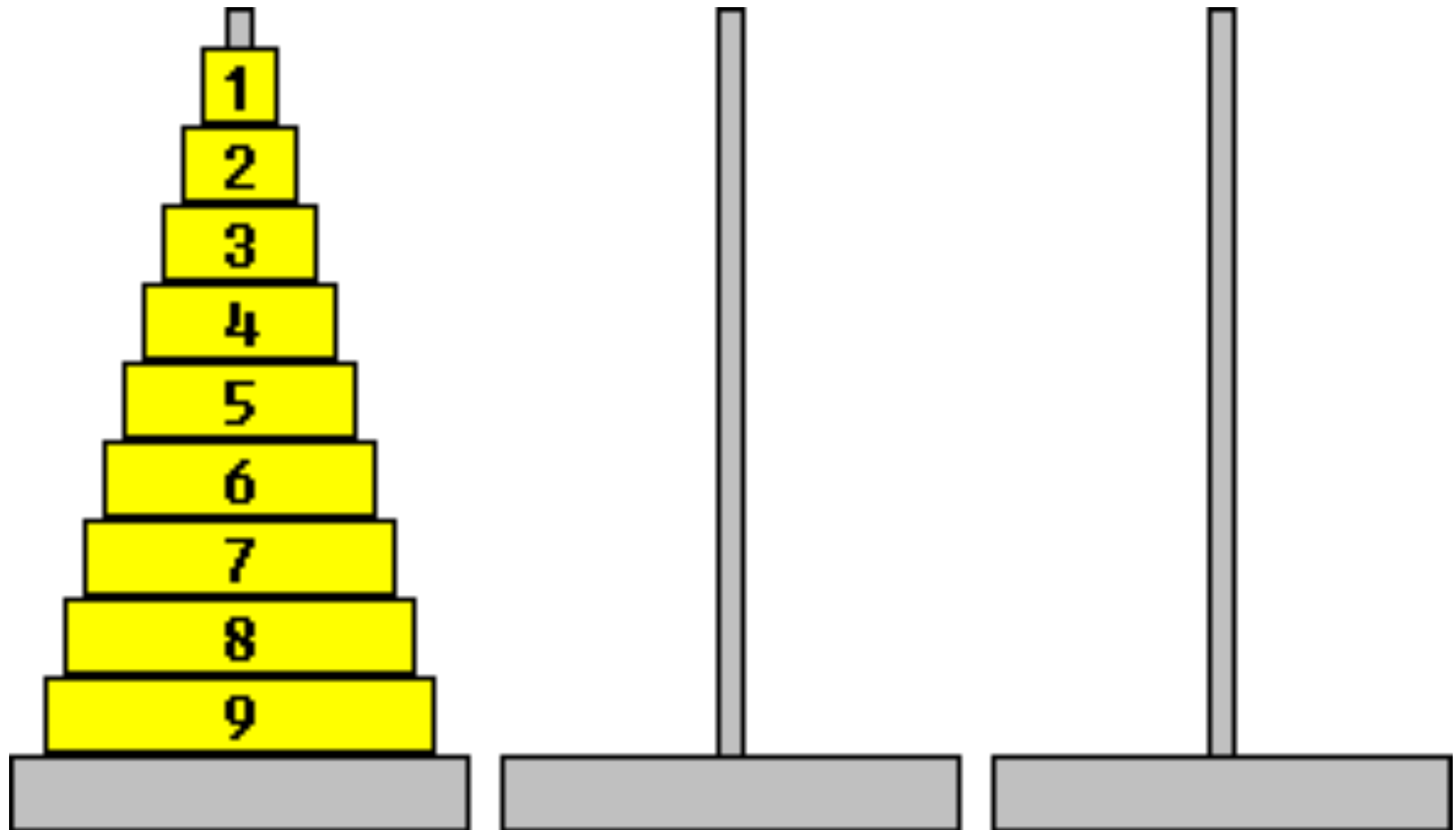
הלולאה מתבצעת $n-2$ פעמים.
נדרש מספר קבוע של משתנים סקלרים.

מגדלי האנוי

האגדה מספרת:

- במקדש [קאשי וישוואנאת](#) (בעיר ואראנסי בהודו) יש חדר גדול ובו שלושה מוטות עליהם מושחלות 64 דיסקיות זהב עגולות שהקוטר שלהן שונה זו מזו.
- כוהנים ברהמינים, בהשראת נבואה עתיקה, עסוקים בהעברת הדיסקיות מהמוט הראשון לשני, בעזרת השלישי.
- בתחילה, הדיסקיות היו מושחלות על המוט הראשון בהתאם לקוטרן - הגדולה ביותר למטה והקטנה ביותר למעלה.
- הדיסקיות מועברות על פי הכללים המחייבים של ברהמה:
 - יש להעביר את הדיסקיות אחת-אחת.
 - מותר להעביר רק דיסקית מראש ערמה אחת לראש ערמה אחרת.
 - אסור להניח דיסקית גדולה יותר על גבי דיסקית קטנה יותר.
- לכשתסתיים העברת הדיסקיות, יבוא העולם לקיצו.
- בגרסה אחרת של האגדה, אלו נזירים בודהיסטים שנמצאים בעיר האנוי (Hà Nội) בווייטנאם.
- משחק הדיסקיות וכנראה גם האגדה הומצאו על ידי המתמטיקאי הצרפתי אדוארד לוקאס ב-1883.
- הערה: קאשי וישוואנאת הוא אחד מ-12 מקדשי הג'יוטירלינגה (Jyotirlinga - ज्योतिर्लिङ्ग) של [שיווה](#). למה במקדש של שיווה (Shiva - शिव) פועלים לפי הכללים של ברהמה (Brahmā - ब्रह्मा)?

המחשה עם 9 דיסקיות



פיתרון ברקורסיה, n דיסקיות

- אם $n == 0$, אין צורך לעשות דבר.
- אחרת (אם $n > 0$),
 - נעביר $n-1$ דיסקיות ממוט 1 למוט 3, בעזרת מוט 2.
 - נעביר את הדיסקית האחרונה ממוט 1 למוט 2.
 - נעביר $n-1$ דיסקיות ממוט 3 למוט 2, בעזרת מוט 1.

<http://towersofhanoi.info/Animate.aspx>

ניתוח

- כמה פעמים קוראים לפונקציה הרקורסיבית? -
- נסמן ב- $T(n)$ את מספר הקריאות שמתבצעות כאשר מפעילים את הפונקציה עבור n דיסקיות.

$$T(0) = 1$$

$$T(n) = 1 + 2T(n-1)$$

- נקבל $T(n) = 2^{n+1} - 1$ (הוכחה באינדוקציה)

(המשך)

- כמה צעדים יש בפיתרון? -
- נסמן ב- $S(n)$ את מספר העברות הדיסקית שמבצעת הקריאה הרקורסיבית עבור n דיסקיות.

$$S(0) = 0$$

$$S(n) = 1 + 2S(n-1)$$

- נקבל $S(n) = 2^n - 1$ (הוכחה באינדוקציה)

קץ העולם

- האם צריך לחשוש שהאגדה נכונה? -
- אפשר להראות שחייבים לבצע לפחות $2^n - 1$ העברות של דיסקית כדי לפתור את הבעייה.
- מספר ההעברות שהברהמינים צריכים לבצע הוא:
 $2^{64} - 1 = 18,446,744,073,709,551,615$
- אם כל העברה לוקחת שנייה, זה ייקח כ-585 מיליארד שנים.
- זה יותר מפי 42 מגיל היקום ויותר מפי 127 מגיל כדור הארץ.

מבני נתונים

טיפוס נתונים מופשטים

- טיפוס נתונים מופשט (abstract data type - ADT) הוא אוסף נתונים בתוספת פעולות שמאפשרות עדכון וגישה לנתונים.
- הארגון הפנימי של הנתונים כדי לאפשר את ביצוע הפעולות ביעילות לא אמור לעניין את המשתמש בטיפוס המופשט.
- הגדרת הממשק (הפעולות) כוללת לפעמים (אבל לא תמיד) התחייבות לסיבוכיות ספציפית של כל פעולה (כפונקציה של גודל האוסף).

מחסנית

מחסנית (stack)

- מחסנית מחזיקה רשימה של אובייקטים.

- הפעולות המותרות:

– הוספת אובייקט `push(object)`

– הוצאת אובייקט `object = pop()`

האובייקט שיוצא הוא האחרון שנכנס
(last-in-first-out, או LIFO).

אם המחסנית ריקה הפעולה שגוייה.

- פעולות אופציונליות:

– בדיקה אם המחסנית ריקה `flag = is_empty()`

– הצצה לאובייקט האחרון `object = peek()`

בפייתון

- אפשר להשתמש במבנה של list כדי ליצור מחסנית:

– אתחול `stack = []`

– הכנסת אובייקט `stack.append(object)`

– הוצאת אובייקט `object = stack.pop()`

דוגמה

```
>>> stack = []
>>> stack.append(1)
>>> stack.append(2)
>>> stack.append(3)
>>> stack.append(4)
>>> stack.pop()
4
>>> stack.pop()
3
>>> stack.append(5)
>>> stack.append(6)
>>> stack.pop()
6
>>> stack.pop()
5
>>> stack.pop()
2
>>> stack.pop()
1
>>> stack.pop()
Traceback (most recent call last):
  File "<pyshell#14>", line 1, in <module>
    stack.pop()
IndexError: pop from empty list
>>>
```

חישוב ערך של ביטוי

$5 * ((6 + 3) / (7 - 2 * 2))$

<i>(initialize)</i>	<i>[]</i>
<i>push(5)</i>	<i>[5]</i>
<i>push(6)</i>	<i>[5, 6]</i>
<i>push(3)</i>	<i>[5, 6, 3]</i>
<i>push(pop() + pop())</i>	<i>[5, 9]</i>
<i>push(7)</i>	<i>[5, 9, 7]</i>
<i>push(2)</i>	<i>[5, 9, 7, 2]</i>
<i>push(2)</i>	<i>[5, 9, 7, 2, 2]</i>
<i>push(pop() * pop())</i>	<i>[5, 9, 7, 4]</i>
<i>b=pop(), a=pop(), push(a-b)</i>	<i>[5, 9, 3]</i>
<i>b=pop(), a=pop(), push(a/b)</i>	<i>[5, 3]</i>
<i>push(pop() * pop())</i>	<i>[15]</i>

מחסנית זמן-ריצה

- בזמן ריצת תוכנית על כל מתודה שמופעלת מכניסים למחסנית את הנתונים הנדרשים:
 - כתובת חזרה (במתודה שקראה לנוכחית)
 - ערכי הפרמטרים שהועברו
 - הקצאת זיכרון עבור המשתנים המקומיים
- בפקודת return מבצעים pop.
- כך מממשים רקורסיה - במחסנית יכולים להיות עותקים רבים של אותה מתודה.

דוגמה

```
>>> def factorial(n):  
    print("-----")  
    print("entering factorial, n =",n)  
    traceback.print_stack()  
    if n == 1:  
        return n  
    else:  
        return n * factorial(n-1)
```

הרצה

```
>>> factorial(3)
-----
entering factorial, n = 3
  File "<string>", line 1, in <module>
  File "/Library/Frameworks/Python.framework/Versions/3.3/lib/python3.3/idlelib/run.py", line 126, in main
    ret = method(*args, **kwargs)
  File "/Library/Frameworks/Python.framework/Versions/3.3/lib/python3.3/idlelib/run.py", line 353, in runcode
    exec(code, self.locals)
  File "<pyshell#92>", line 1, in <module>
  File "<pyshell#91>", line 4, in factorial
-----
entering factorial, n = 2
  File "<string>", line 1, in <module>
  File "/Library/Frameworks/Python.framework/Versions/3.3/lib/python3.3/idlelib/run.py", line 126, in main
    ret = method(*args, **kwargs)
  File "/Library/Frameworks/Python.framework/Versions/3.3/lib/python3.3/idlelib/run.py", line 353, in runcode
    exec(code, self.locals)
  File "<pyshell#92>", line 1, in <module>
  File "<pyshell#91>", line 8, in factorial
  File "<pyshell#91>", line 4, in factorial
-----
entering factorial, n = 1
  File "<string>", line 1, in <module>
  File "/Library/Frameworks/Python.framework/Versions/3.3/lib/python3.3/idlelib/run.py", line 126, in main
    ret = method(*args, **kwargs)
  File "/Library/Frameworks/Python.framework/Versions/3.3/lib/python3.3/idlelib/run.py", line 353, in runcode
    exec(code, self.locals)
  File "<pyshell#92>", line 1, in <module>
  File "<pyshell#91>", line 8, in factorial
  File "<pyshell#91>", line 8, in factorial
  File "<pyshell#91>", line 4, in factorial
6
>>>
```

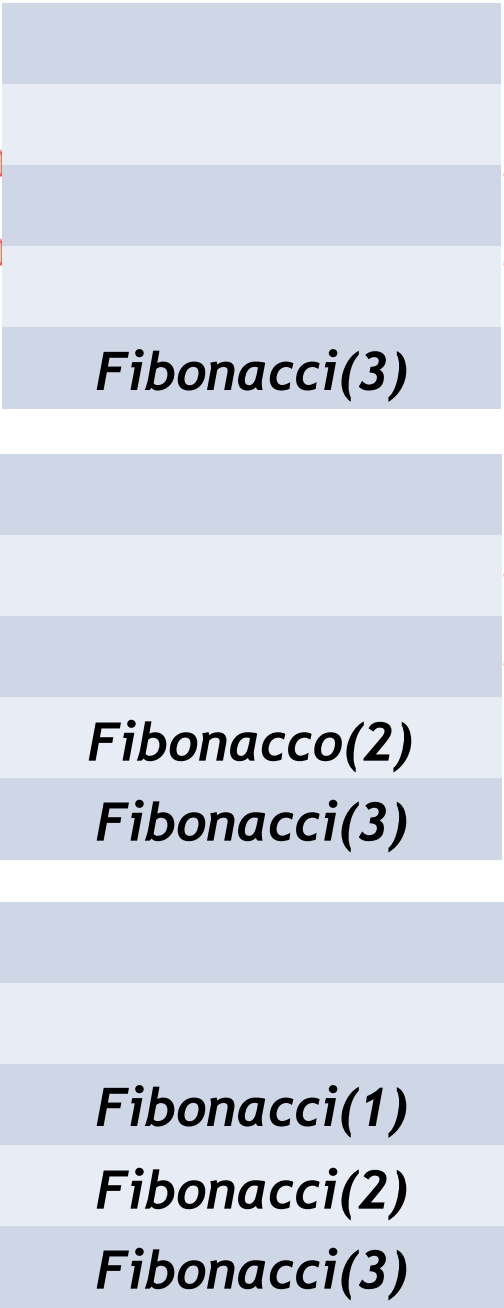
עוד דוגמה

```
>>> def Fibonacci(n):  
    print("-----")  
    print("entering Fibonacci, n =", n)  
    traceback.print_stack()  
    if n <= 1:  
        return n  
    else:  
        return Fibonacci(n-1) + Fibonacci(n-2)
```

```
>>> Fibonacci(3)  
-----
```

הרצה

```
>>> Fibonacci(3)
-----
entering Fibonacci, n = 3
File "<string>", line 1, in <module>
File "/Library/Frameworks/Python.framework/Versions/3.3/lib/python3.3/idlelib/main.py", line 141, in main
    ret = method(*args, **kwargs)
File "/Library/Frameworks/Python.framework/Versions/3.3/lib/python3.3/idlelib/run.py", line 111, in runcode
    exec(code, self.locals)
File "<pyshell#101>", line 1, in <module>
File "<pyshell#100>", line 4, in Fibonacci
-----
entering Fibonacci, n = 2
File "<string>", line 1, in <module>
File "/Library/Frameworks/Python.framework/Versions/3.3/lib/python3.3/idlelib/main.py", line 141, in main
    ret = method(*args, **kwargs)
File "/Library/Frameworks/Python.framework/Versions/3.3/lib/python3.3/idlelib/run.py", line 111, in runcode
    exec(code, self.locals)
File "<pyshell#101>", line 1, in <module>
File "<pyshell#100>", line 8, in Fibonacci
File "<pyshell#100>", line 4, in Fibonacci
-----
entering Fibonacci, n = 1
File "<string>", line 1, in <module>
File "/Library/Frameworks/Python.framework/Versions/3.3/lib/python3.3/idlelib/main.py", line 141, in main
    ret = method(*args, **kwargs)
File "/Library/Frameworks/Python.framework/Versions/3.3/lib/python3.3/idlelib/run.py", line 111, in runcode
    exec(code, self.locals)
File "<pyshell#101>", line 1, in <module>
File "<pyshell#100>", line 8, in Fibonacci
File "<pyshell#100>", line 8, in Fibonacci
File "<pyshell#100>", line 4, in Fibonacci
```



The call stack visualization on the right shows the sequence of function calls during the execution of `Fibonacci(3)`. It consists of three frames, each with a light blue header and a dark blue footer containing the function name and its arguments. The frames are stacked vertically, with the most recent call at the top.

- Top frame: `Fibonacci(3)`
- Middle frame: `Fibonacci(2)`
- Bottom frame: `Fibonacci(3)`

(המשך)

```
-----
entering Fibonacci, n = 0
File "<string>", line 1, in <module>
File "/Library/Frameworks/Python.framework/Versions/3.3/lib/python3.3/i 6, in main
    ret = method(*args, **kwargs)
File "/Library/Frameworks/Python.framework/Versions/3.3/lib/python3.3/i 3, in runcode
    exec(code, self.locals)
File "<pyshell#101>", line 1, in <module>
File "<pyshell#100>", line 8, in Fibonacci
File "<pyshell#100>", line 8, in Fibonacci
File "<pyshell#100>", line 4, in Fibonacci
-----
entering Fibonacci, n = 1
File "<string>", line 1, in <module>
File "/Library/Frameworks/Python.framework/Versions/3.3/lib/python3.3/i 6, in main
    ret = method(*args, **kwargs)
File "/Library/Frameworks/Python.framework/Versions/3.3/lib/python3.3/i 3, in runcode
    exec(code, self.locals)
File "<pyshell#101>", line 1, in <module>
File "<pyshell#100>", line 8, in Fibonacci
File "<pyshell#100>", line 4, in Fibonacci
2
>>>
```

Fibonacci(0)

Fibonacci(2), 1

Fibonacci(3)

Fibonacci(1)

Fibonacci(3), 0+1

מימוש מחסנית בעזרת מערך

- כל פעולה על המחסנית ממומשת על ידי מספר קבוע של פעולות בסיסיות במחשב (לא תלוי בגודל המחסנית).
- גודל המחסנית מוגבל - אפשר להתגבר על זה על ידי הגדלת המערך אם ממלאים אותו.

הקוד

```
>>> class Stack:
    def __init__(self, size = 100):
        self.stack = [None] * size
        self.size = size
        self.top = 0 # first empty entry is top of stack
    def push(self, object):
        if self.top >= self.size:
            raise RuntimeError('My stack is full')
        self.stack[self.top] = object
        self.top += 1
    def pop(self):
        if self.top <= 0:
            raise RuntimeError('My stack is empty')
        self.top -= 1
        return self.stack[self.top]
```

דוגמת הרצה

```
>>> stack = Stack(3)
>>> stack.push(1)
>>> stack.push(2)
>>> stack.push(3)
>>> stack.pop()
3
>>> stack.pop()
2
>>> stack.push(4)
>>> stack.push(5)
>>> stack.pop()
5
>>> stack.push(6)
>>> stack.push(7)
Traceback (most recent call last):
  File "<pyshell#142>", line 1, in <module>
    stack.push(7)
  File "<pyshell#131>", line 8, in push
    raise RuntimeError('My stack is full')
RuntimeError: My stack is full
>>>
```

מימוש בעזרת הספרייה של פייתון

```
class Stack1:
    def __init__(self):
        self.__lst = []

    def push(self, item):
        self.__lst.append(item)

    def pop(self):
        return self.__lst.pop()

    def is_empty(self):
        return len(self.__lst)==0
```

```
stack = Stack1()
stack.push(1)
stack.pop()
stack.push(2)
print(stack.is_empty())
```

תור

תור (queue)

- כמו מחסנית, תור מחזיק רשימה של אובייקטים.
- הפעולות המותרות:

– הוספת אובייקט `enqueue(object)`

– הוצאת אובייקט `object = deque()`

האובייקט שיוצא הוא הראשון שנכנס

(first-in-first-out, או FIFO).

אם התור ריק הפעולה שגוייה.

- פעולות אופציונליות:

– בדיקה אם התור ריק `flag = is_empty()`

בפייתון

- בספריה של פייתון יש מחלקה בשם `queue.Queue` שמממשת תור (הכפילות בשם היא כי יש תחת `queue` עוד סוגים של תורים):

– אתחול `q = queue.Queue()`

– הוספת אובייקט לתור `q.put(obj)`

– הוצאת אובייקט מהתור `obj = q.get()`

מימוש תור בעזרת מערך

- כל פעולה דורשת מספר קבוע של פעולות בסיסיות במחשב (לא תלוי בגודל התור).
- גודל התור מוגבל.
- נשתמש בשני אינדקסים:
 - ראש התור יסומן ב-head
 - סוף התור יסומן ב-tail. זה המקום הפנוי הבא להוסיף אובייקט.
 - בתור ריק $head == tail$.
 - נקדם את head ו-tail באופן ציקלי.

הקוד

```
>>> class Queue:
    # no error handling
    def __init__(self, size = 100):
        self.queue = [None] * size
        self.size = size
        self.head = self.tail = 0
    def enqueue(self, obj):
        self.queue[self.tail] = obj
        self.tail = (self.tail + 1) % self.size
    def deque(self):
        obj = self.queue[self.head]
        self.head = (self.head + 1) % self.size
        return obj
```

דוגמת הרצה

```
>>> queue = Queue(5)
>>> queue.enqueue(1)
>>> queue.enqueue(2)
>>> queue.enqueue(3)
>>> queue.enqueue(4)
>>> queue.dequeue()
1
>>> queue.dequeue()
2
>>> queue.enqueue(5)
>>> queue.enqueue(6)
>>> queue.dequeue()
3
>>>
```

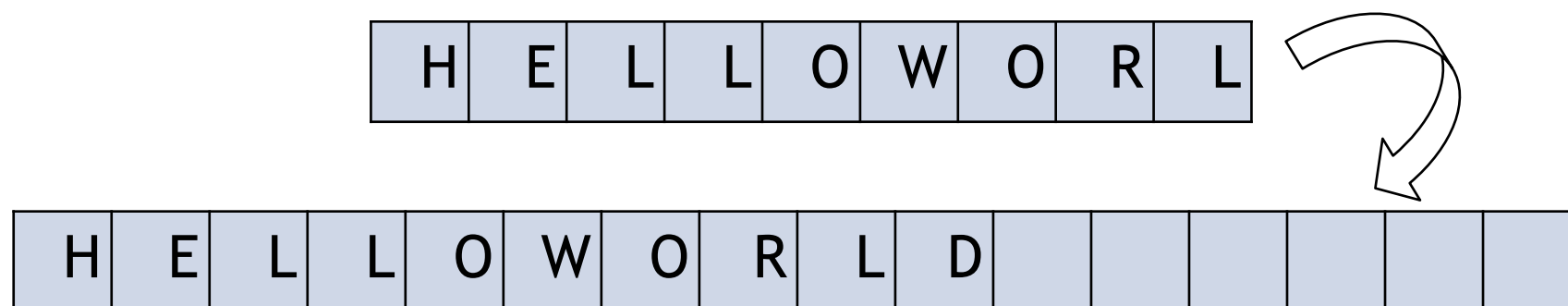
רשימה מקושרת

הרשימות (lists) של פייתון בזיכרון

- בפייתון רשימה (list) מוחזקת ברצף בזיכרון:



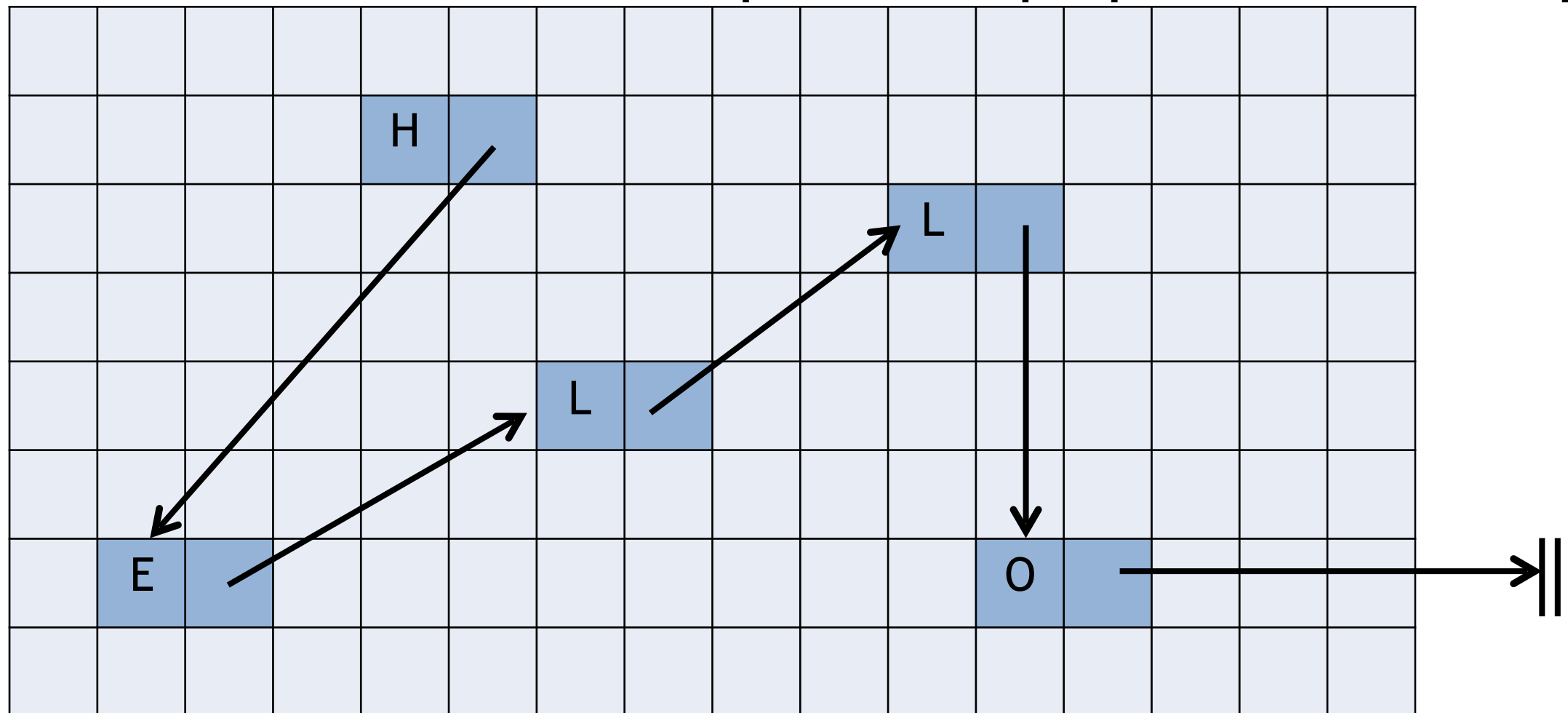
- אם אנחנו רוצים להגדיל את הרשימה, לעיתים מזומנות צריך להקצות מקום חדש ולהעתיק אותה:



- בפייתון זה נעשה לעיתים מזומנות מאחורי הקלעים.

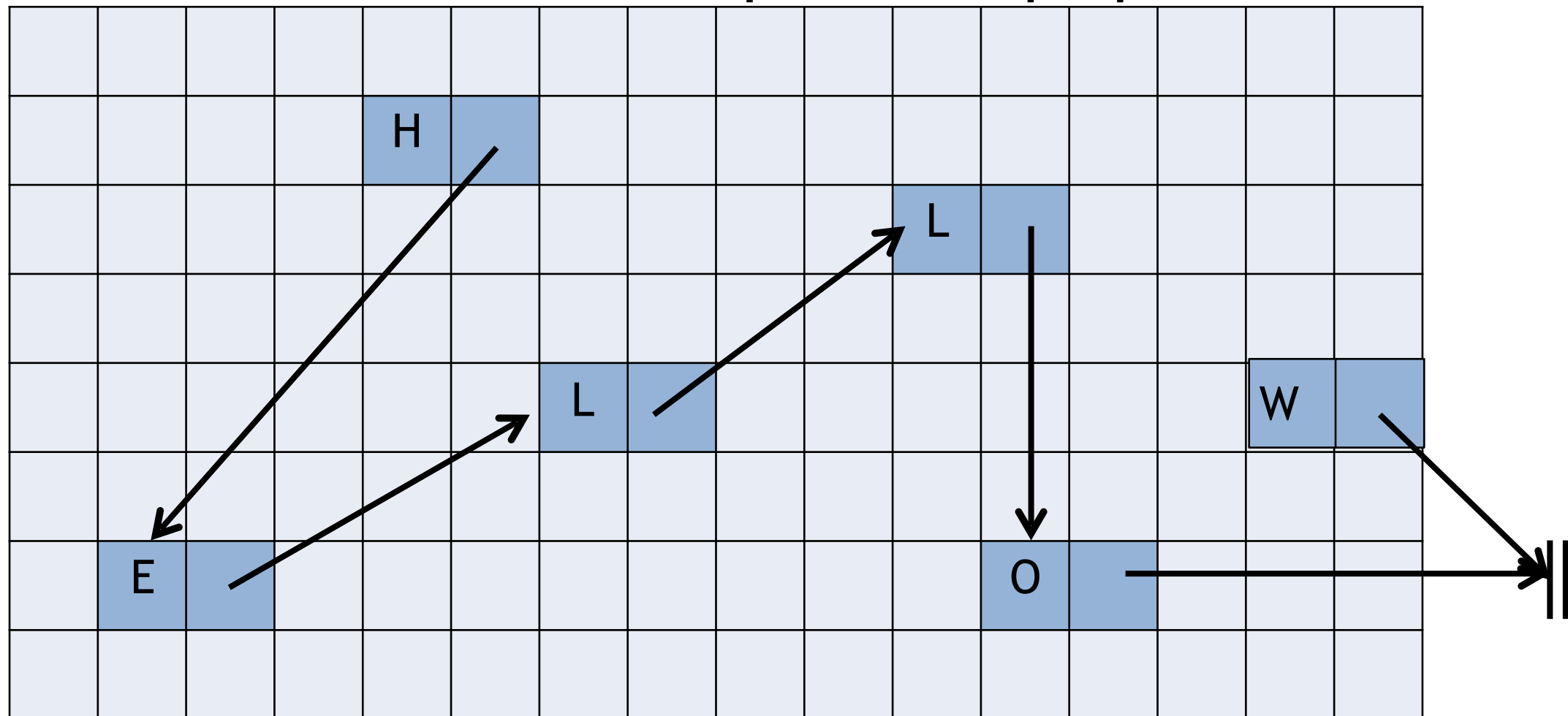
רשימות מקושרות

- מוחזקות בהרבה קטעי זיכרון במקום אחד
- משרשרים את קטעי הזיכרון—כל קטע מחזיק את מיקום הקטע הבא
- כך אפשר להוסיף קטעים בקלות:



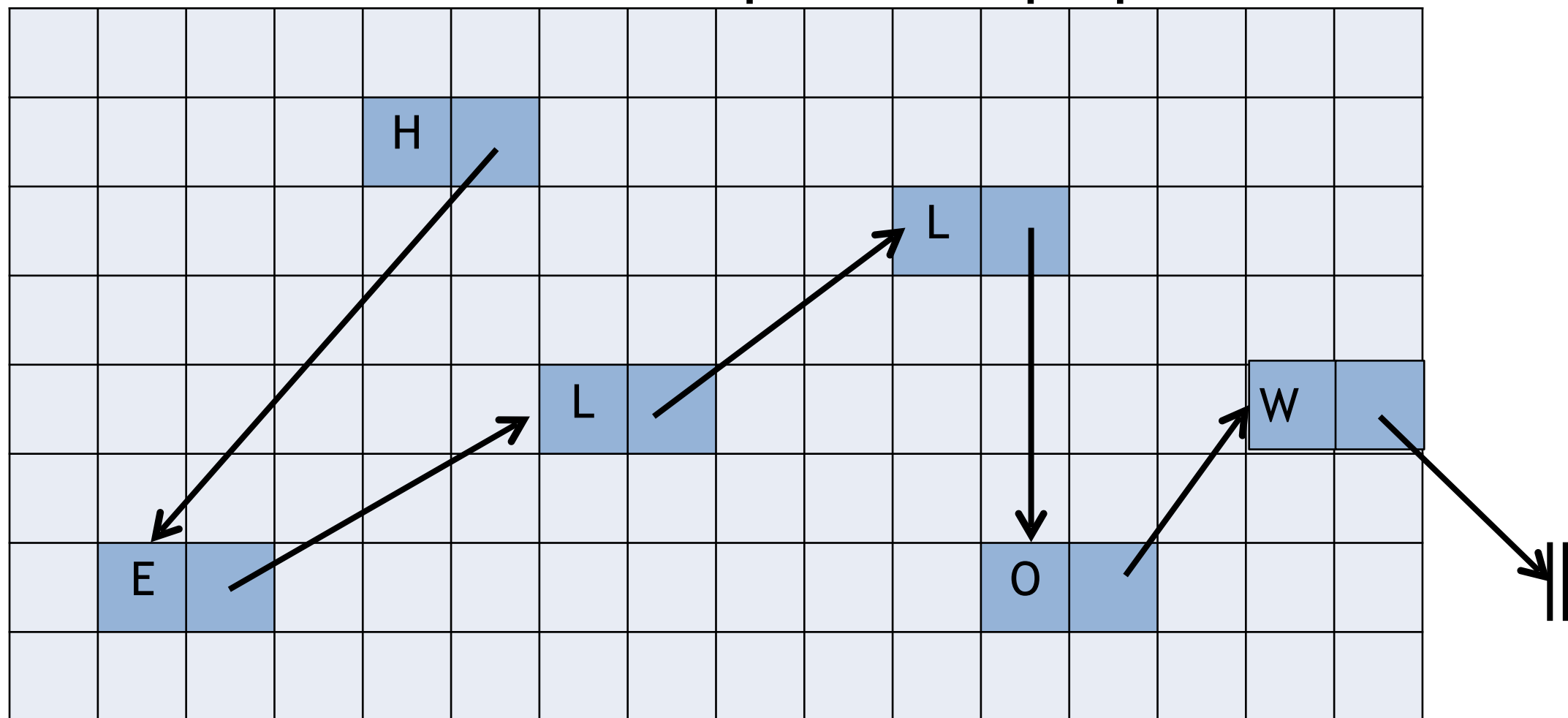
רשימות מקושרות

- מוחזקות בהרבה קטעי זיכרון במקום אחד
- משרשרים את קטעי הזיכרון—כל קטע מחזיק את מיקום הקטע הבא
- כך אפשר להוסיף קטעים בקלות:

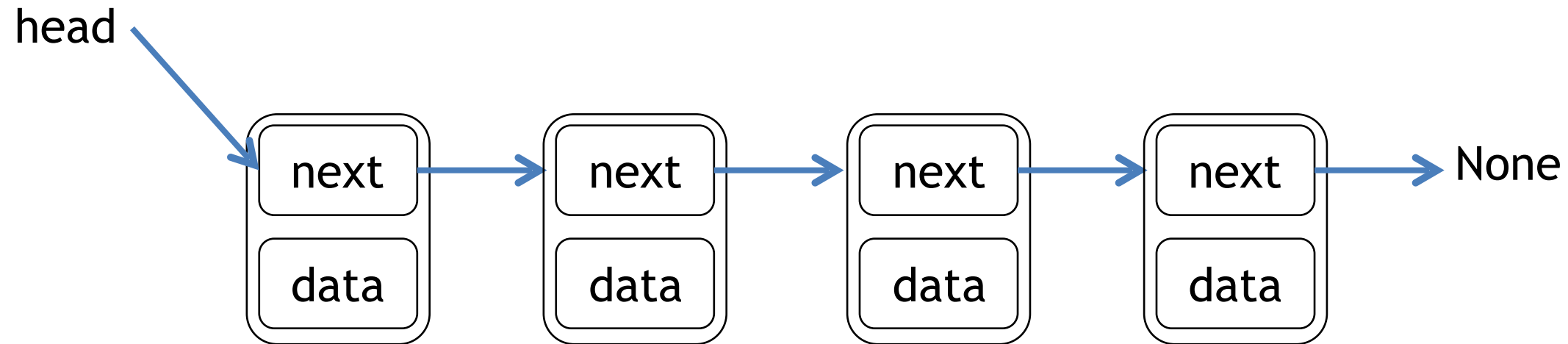


רשימות מקושרות

- מוחזקות בהרבה קטעי זיכרון במקום אחד
- משרשרים את קטעי הזיכרון—כל קטע מחזיק את מיקום הקטע הבא
- כך אפשר להוסיף קטעים בקלות:



רשימה מקושרת



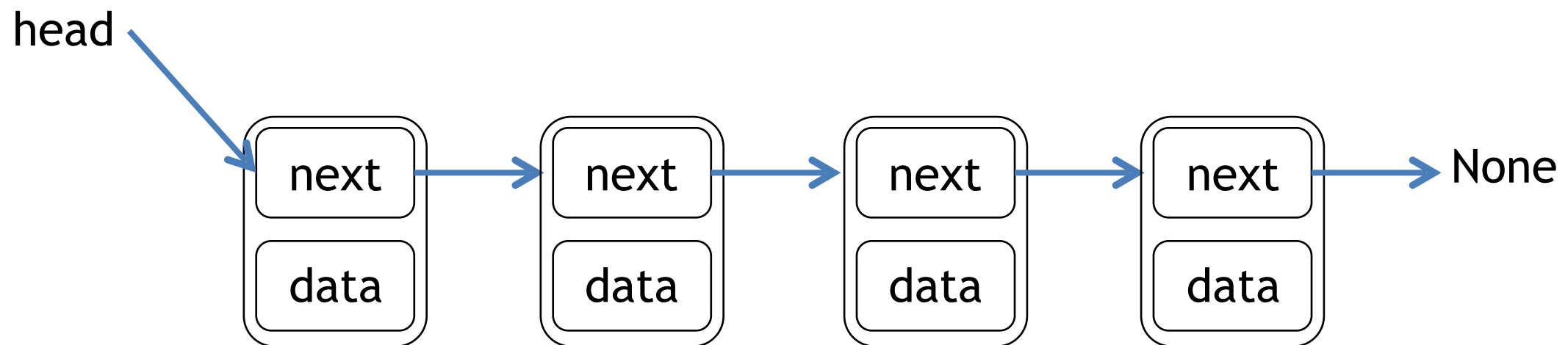
```
class Node:
    def __init__(self, data=None, next=None):
        self.data = data
        self.next = next
```

איך עובדים איתן?

- גישה לנתונים ברשימה:

```
head.data  
head.next.data  
head.next.next.data  
...
```

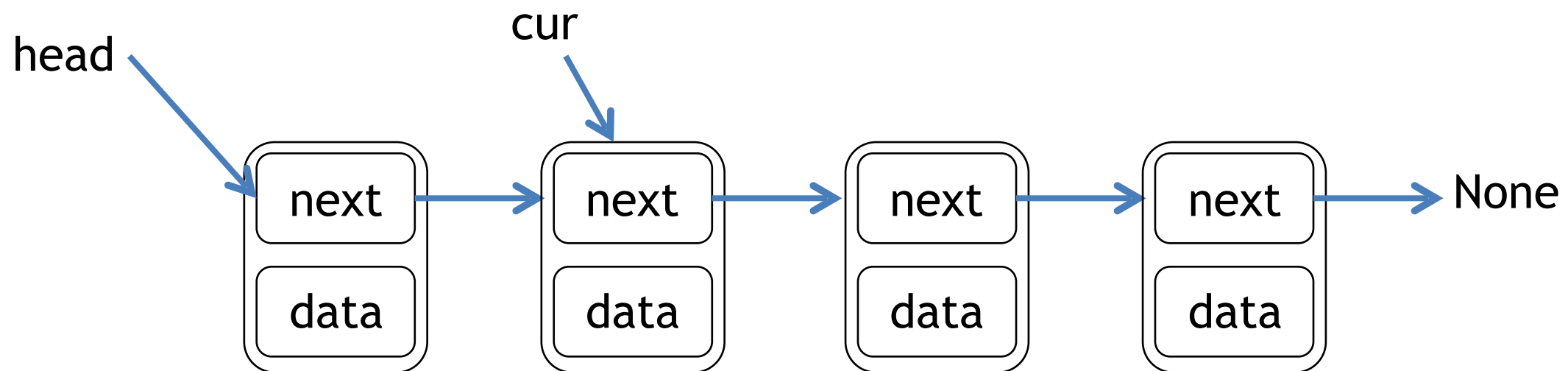
- זה מסורבל:



(המשך)

- במקום זאת, בדרך כלל מחזיקים מראה מקום נייד:

```
cur = head  
print(cur.data)  
cur = cur.next  
print(cur.data)  
...
```



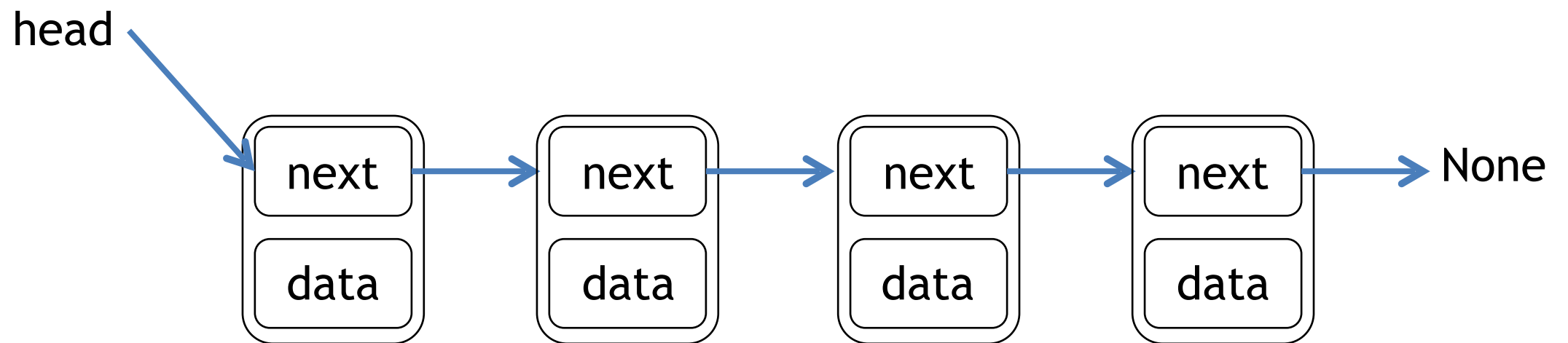
מעבר על הרשימה כולה

- למשל, כדי להדפיס רשימה:

```
cur = head
while cur!=None:
    print(cur.data)
    cur = cur.next
```

הוספת איבר לרשימה

- בראש הרשימה:



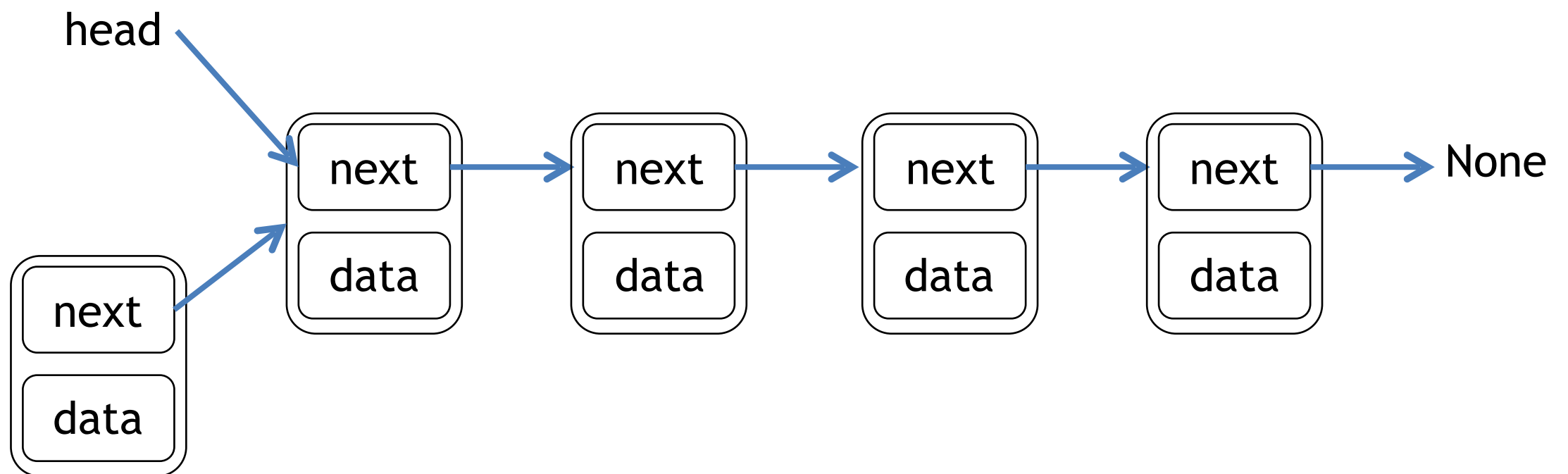
```
new_node = Node("new val", head)
head = new_node
```

לחילופין:

```
head = Node("new val", head)
```

הוספת איבר לרשימה

- בראש הרשימה:



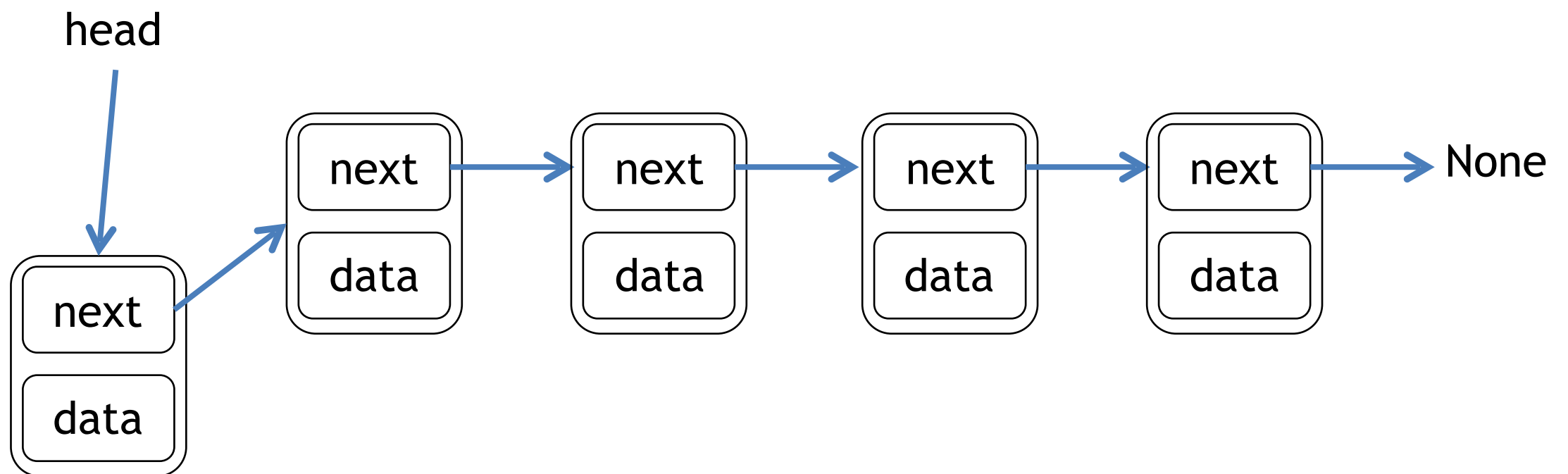
```
new_node = Node("new val", head)
head = new_node
```

לחילופין:

```
head = Node("new val", head)
```

הוספת איבר לרשימה

- בראש הרשימה:



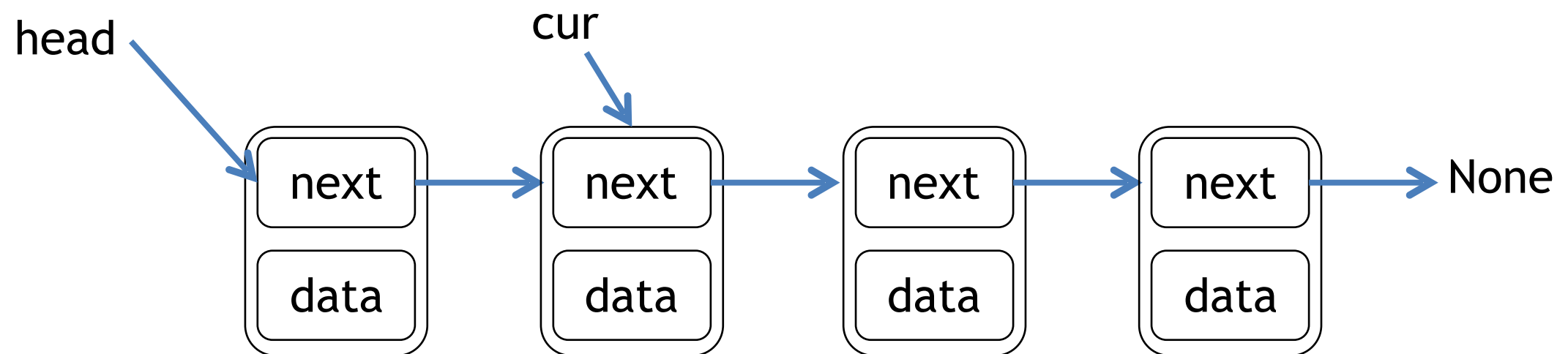
```
new_node = Node("new val", head)
head = new_node
```

לחילופין:

```
head = Node("new val", head)
```

הוספת איבר לרשימה

- הוספה במקום `cur`:

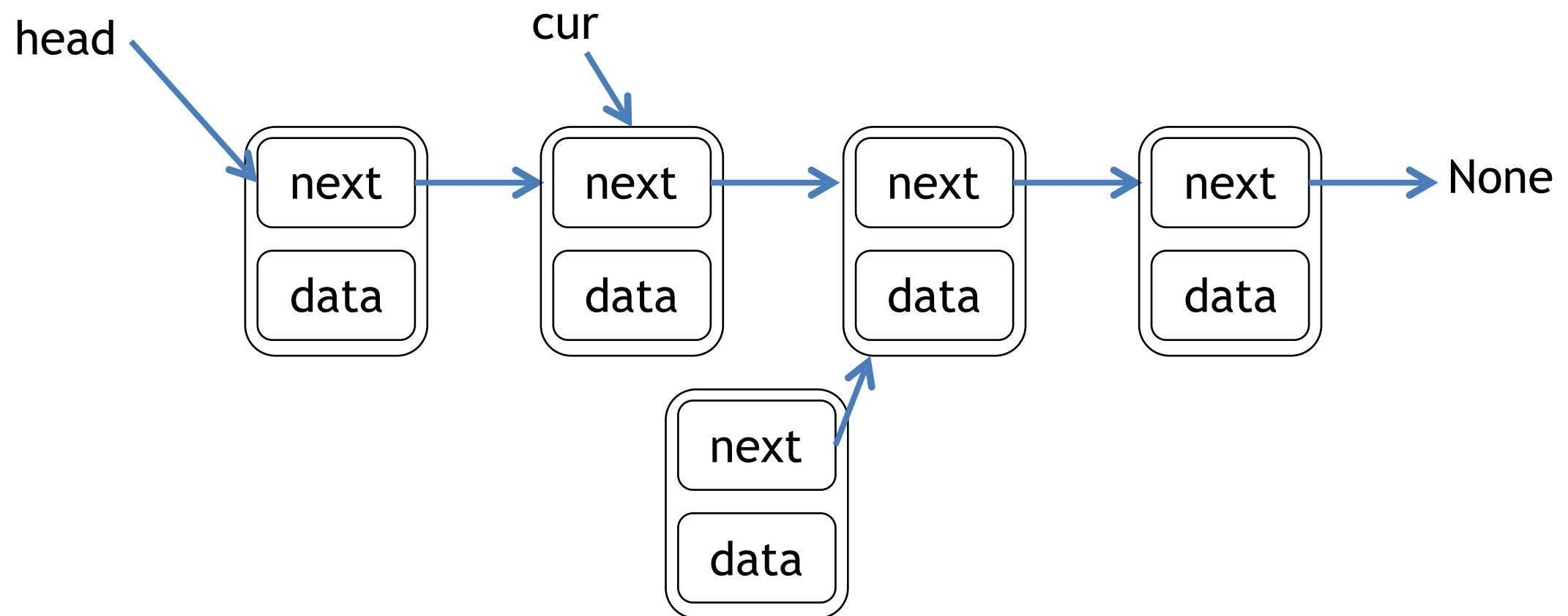


```
cur.next = Node("some val", cur.next)
```

- שימו לב: זה דורש מספר קבוע של פעולות.
לעומת זאת, הוספת איבר ב-`list` דורשת לעיתים העתקה של כל הרשימה.

הוספת איבר לרשימה

- הוספה במקום `cur`:

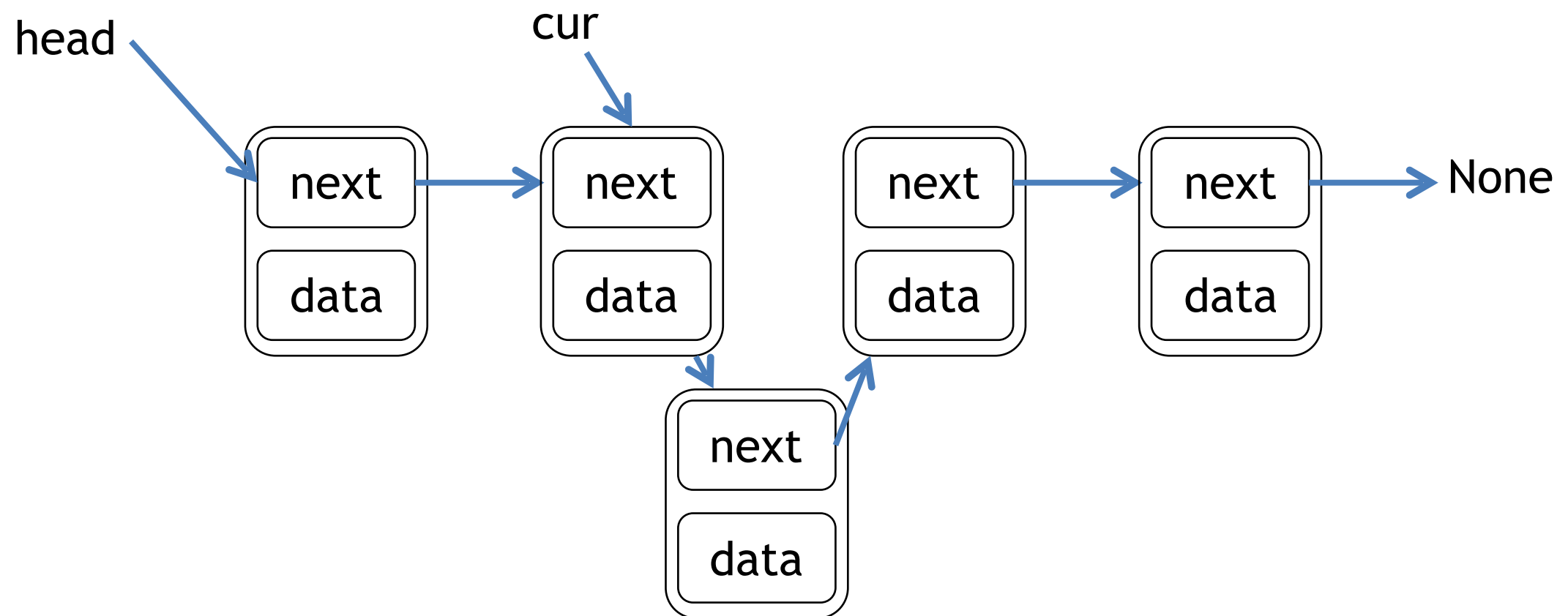


```
cur.next = Node("some val", cur.next)
```

- שימו לב: זה דורש מספר קבוע של פעולות.
לעומת זאת, הוספת איבר ב-`list` דורשת לעיתים העתקה של כל הרשימה.

הוספת איבר לרשימה

- הוספה במקום `cur`:

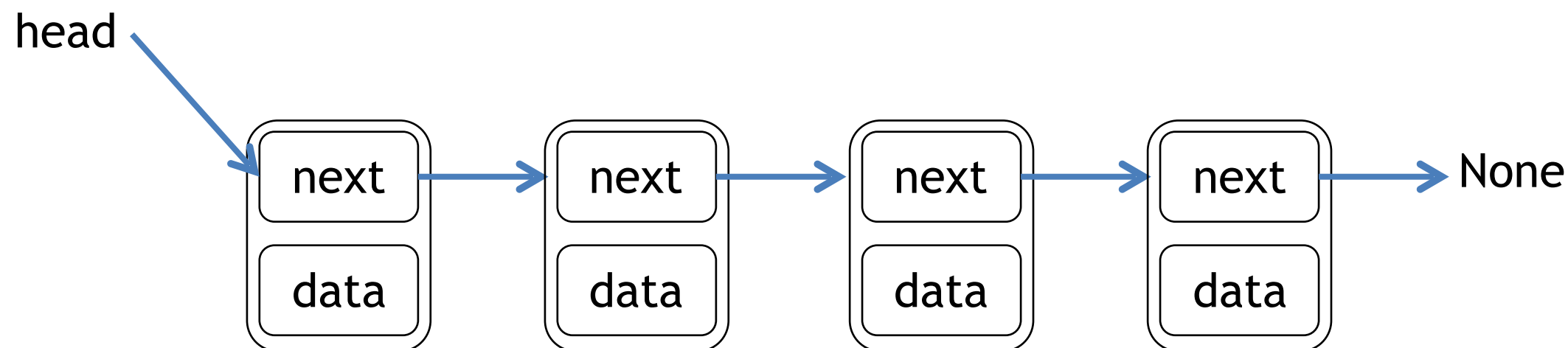


```
cur.next = Node("some val", cur.next)
```

- שימו לב: זה דורש מספר קבוע של פעולות.
לעומת זאת, הוספת איבר ב-`list` דורשת לעיתים העתקה של כל הרשימה.

הסרת איבר מהרשימה

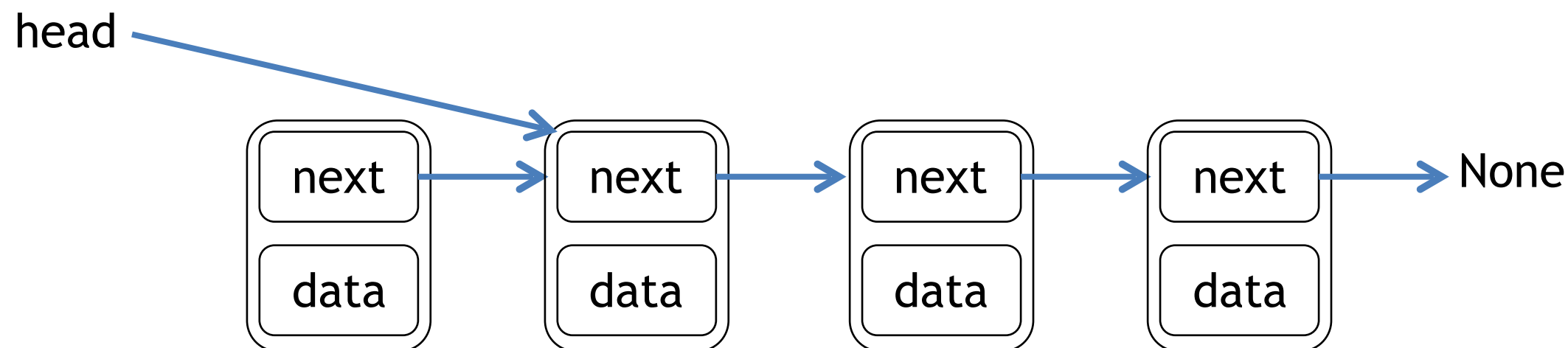
- הסרת האיבר הראשון:



```
head = head.next
```

הסרת איבר מהרשימה

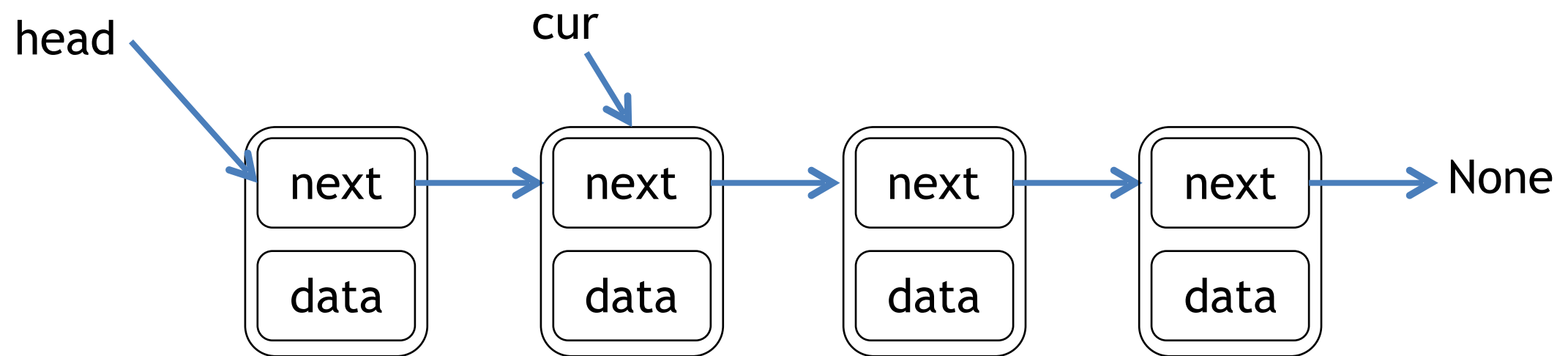
- הסרת האיבר הראשון:



```
head = head.next
```

(המשך)

- הסרת האיבר הראשון אחרי `cur`:

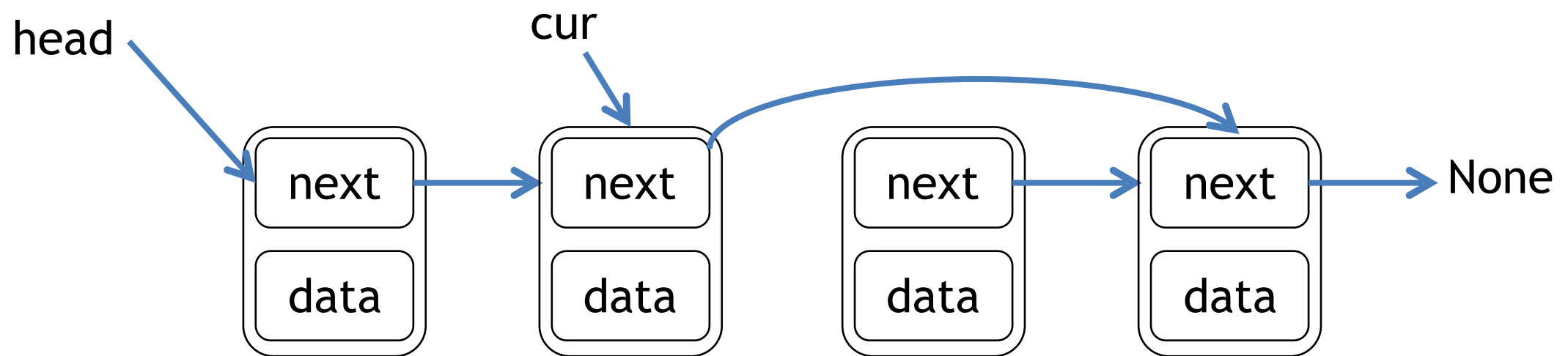


```
cur.next = cur.next.next
```

- מה נעשה אם נרצה להסיר את `cur`?

(המשך)

- הסרת האיבר הראשון אחרי `cur`:



```
cur.next = cur.next.next
```

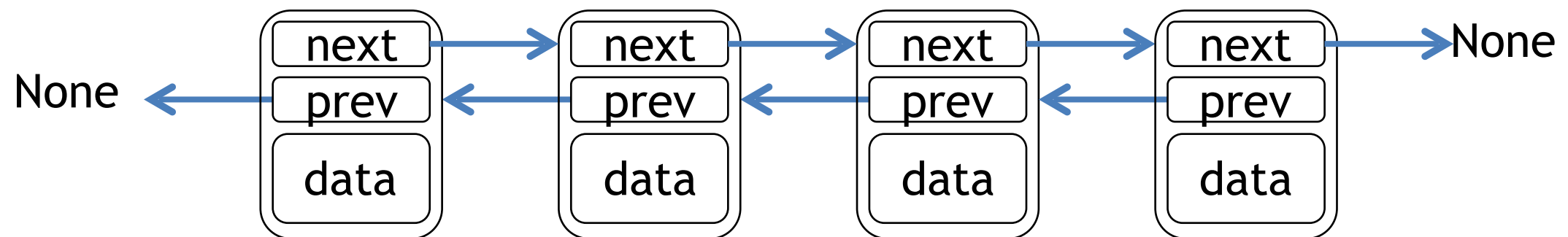
- מה נעשה אם נרצה להסיר את `cur`?

חלק מהפעולות על רשימות יקרות יותר מאשר
ב-list:

- איך מגיעים לאיבר לפני cur?
 - איך מגיעים לאיבר ה-k-י ברשימה?
 - איך מדפיסים את הרשימה בסדר הפוך?
- ומה אם נאסר עלינו להקצות יותר ממספר קבוע של
תאי זיכרון נוספים?

רשימות דו-קשירות

- אפשר ליצור רשימות עם מצביעים בשני הכיוונים:



- קל יותר לבצע צעד אחד אחורה
- פעולות הכנסה והוצאה של איברים מסובכות מעט יותר
- הרשימה דורשת יותר מקום בזיכרון

מימוש מחסנית על ידי רשימה מקושרת

```
class Node:
    def __init__(self, data=None, next=None):
        self.data = data
        self.next = next
```

```
class Stack:
    def __init__(self):
        self.__head = None

    def push(self, item):
        self.__head = Node(item, self.__head)

    def pop(self):
        result = self.__head.data
        self.__head = self.__head.next
        return result

    def is_empty(self):
        return self.__head == None
```

שיפור המחלקה

- המרה נוחה של הרשימה למחרוזת:

```
def __str__(self):  
    cur = self.__head  
    result = ""  
    while cur!=None:  
        result = result+str(cur.data)  
        if cur.next!=None:  
            result = result+","  
        cur = cur.next  
    return "[" + result + "]"
```

רקורסיה ורשימות מקושרות

- לעיתים מזומנות מעבר ברקורסיה על רשימה מקושרת הוא טבעי

- לרשימה יש מבנה רקורסיבי: אם מסירים את ראש הרשימה, מה שנותר הוא רשימה מקושרת.

- דוגמה:

```
def get_size(self):  
    def recursive_size(list_head):  
        if list_head==None:  
            return 0  
        else:  
            return recursive_size(list_head.next)+1  
  
    return recursive_size(self.__head)
```

מציאת איבר ברשימה

- לדוגמה, במחסנית שהגדרנו:

```
def find_ver1(self, item):  
    cur = self.__head  
    while cur:  
        if cur.data==item:  
            return True  
        cur=cur.next  
    return False  
  
def find_ver2(self, item):  
    return _find_helper(self.__head, item)
```

גרסה אחרת

• ברקורסיה:

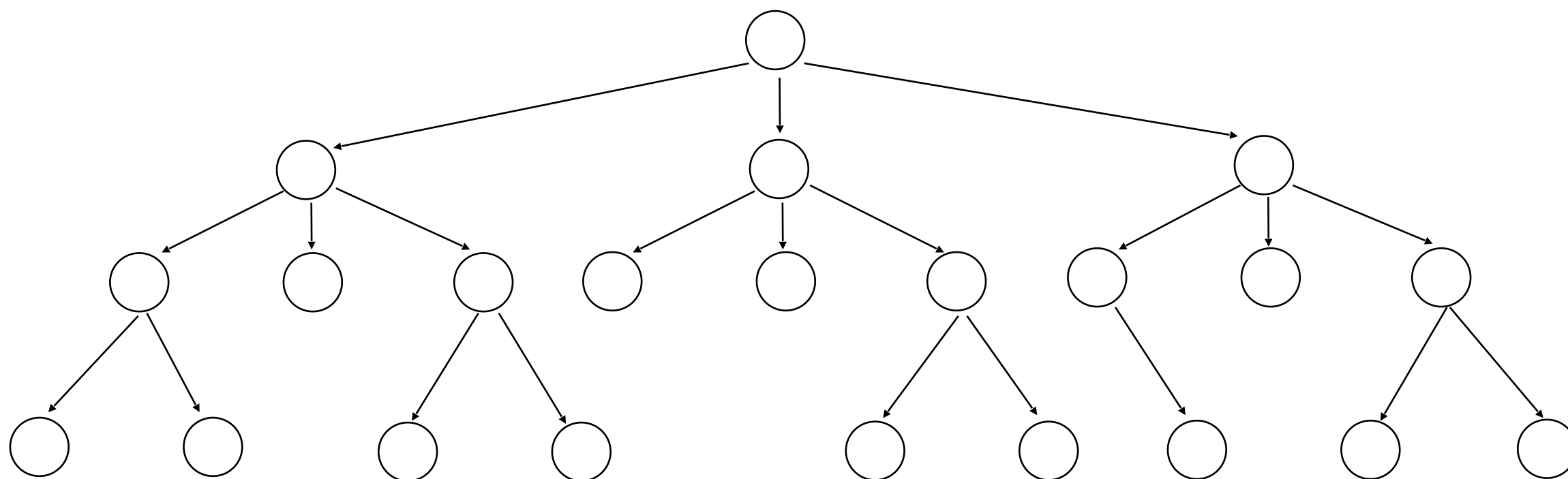
```
def find_ver2(self, item):  
    return _find_helper(self.__head, item)  
  
def _find_helper(node, item):  
    if not node:  
        return False  
  
    if node.data == item:  
        return True  
    else:  
        return _find_helper(node.next, item)
```

עץ

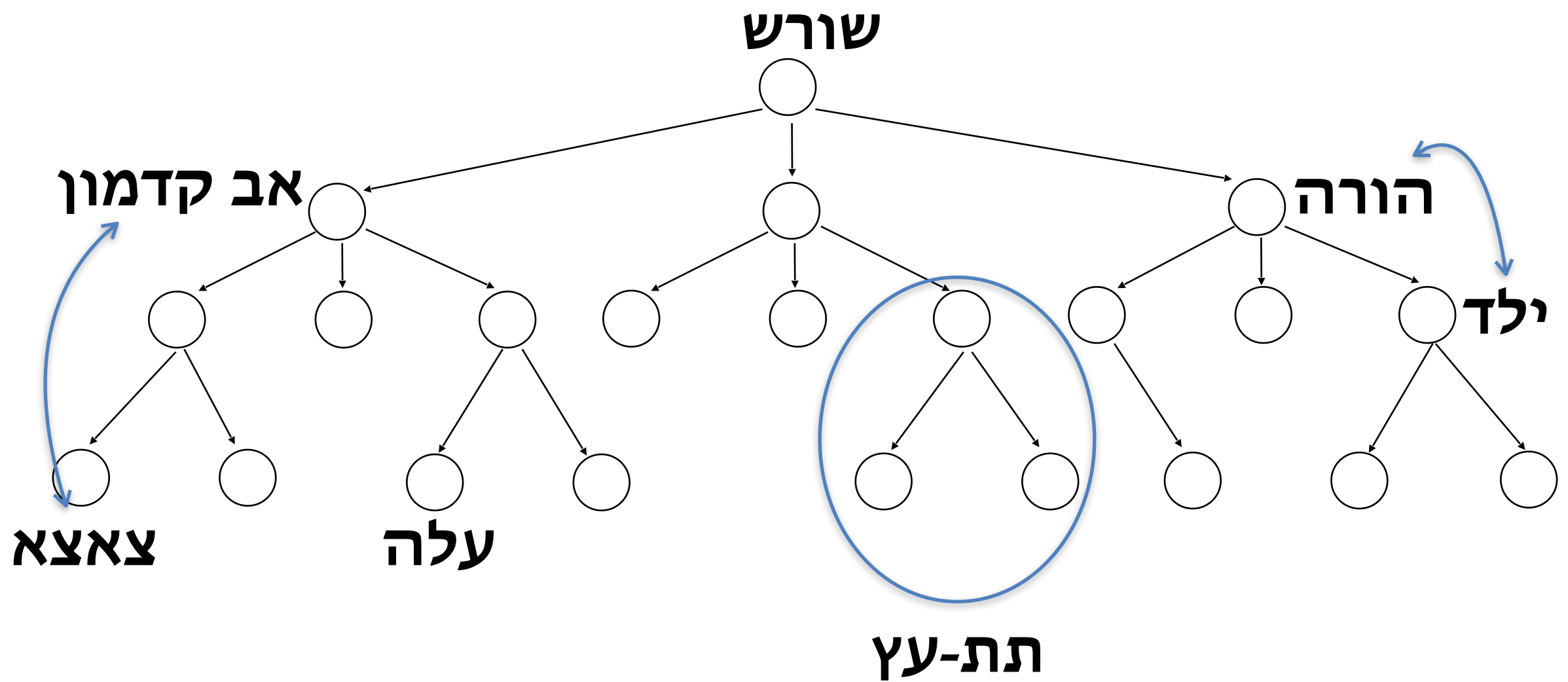
עצים

- ברשימה מקושרת כל צומת מחזיק מצביע בודד.
- אפשר להחזיק בצומת יותר ממצביע אחד וכך לייצר מבנים מורכבים יותר.
- עץ הוא מבנה נתונים שמוגדר באינדוקציה כך:
 - צומת ללא מצביעים הוא עץ.
 - מבנה של צומת (שורש) עם מצביעים לעצים (קטנים יותר) שהם זרים זה לזה ואינם מכילים את השורש הוא עץ.

תרשים



תרשים



עצים בינאריים

- עץ שבו לכל צומת יש לכל היותר שני ילדים נקרא עץ בינארי.
- זו מחלקה שאפשר לבנות ממנה עצים בינאריים:

```
>>> class BinaryNode:
    def __init__(self, data, left_child = None, right_child = None):
        self.data = data
        self.left = left_child
        self.right = right_child
```

```
>>>
```

סריקת עץ בינארי

- סריקה postorder, הגדרה רקורסיבית:
 - אם לשורש יש ילד שמאלי, סורקים את תת-העץ שלו ראשון.
 - אח"כ, אם לשורש יש ילד ימני, סורקים את תת-העץ שלו.
 - לבסוף, מדפיסים את השדה data של השורש.
- סריקה preorder: קודם מדפיסים את השדה data של השורש ואז סורקים את תתי-העצים.
- סריקה inorder: ילד שמאלי, שורש, ילד ימני.

הקוד

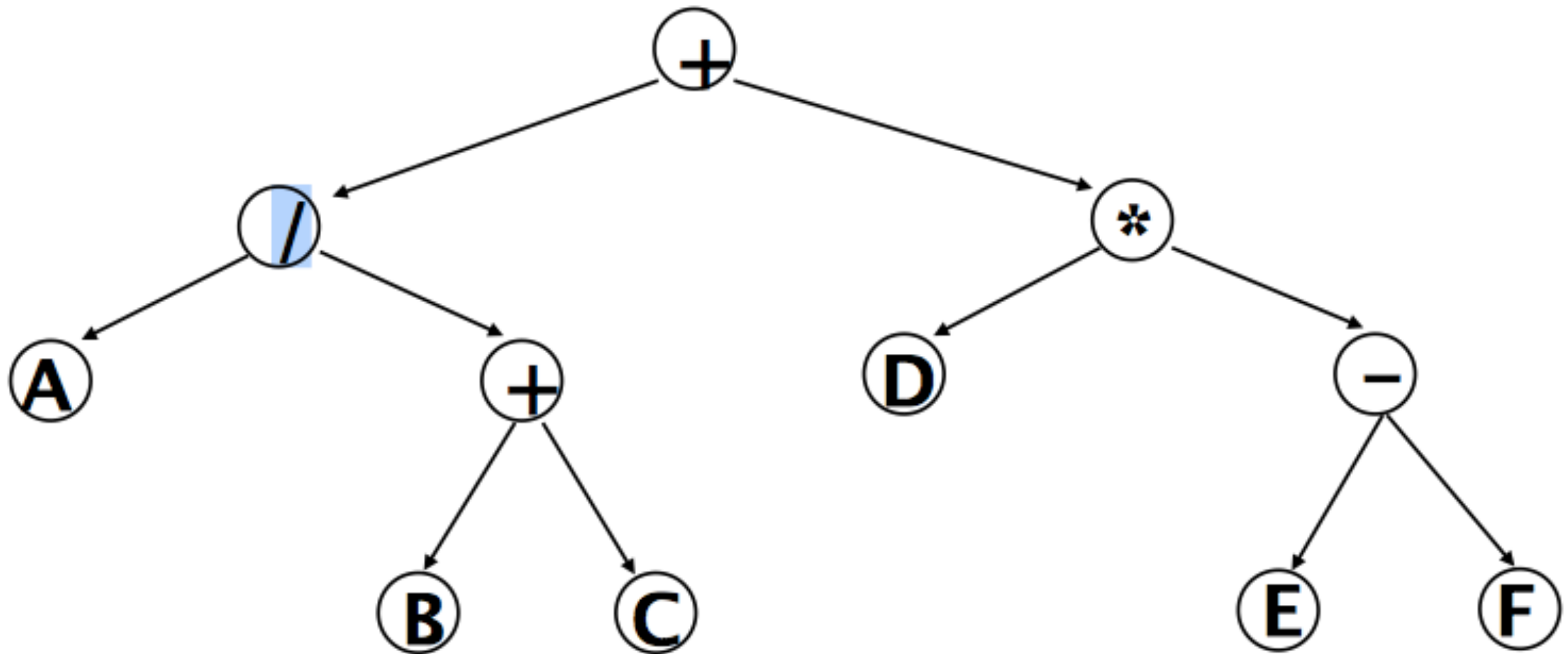
```
>>> class BinaryNode:
    def __init__(self, data, left_child = None, right_child = None):
        self.data = data
        self.left = left_child
        self.right = right_child
    def postorder(self):
        if self.left != None:
            self.left.postorder()
        if self.right != None:
            self.right.postorder()
        print(self.data)
```

מימוש אלטרנטיבי

```
>>> class BinaryNode:
    def __init__(self, data, left = None, right = None):
        self.data = data
        self.left = left
        self.right = right
    def postorder(self):
        if self.left != None:
            for data in self.left.postorder():
                yield data
        if self.right != None:
            for data in self.right.postorder():
                yield data
        yield self.data
```

שימוש

- אפשר להחזיק בעץ בינארי ביטוי חשבוני.
- לדוגמה, העץ של הביטוי: $a / (b + c) + d * (e - f)$ יראה כך:



```

>>> a = BinaryNode("a")
>>> b = BinaryNode("b")
>>> c = BinaryNode("c")
>>> d = BinaryNode("d")
>>> e = BinaryNode("e")
>>> f = BinaryNode("f")
>>> bplusc = BinaryNode("+", b, c)
>>> eminusf = BinaryNode("-", e, f)
>>> adivsum = BinaryNode("/", a, bplusc)
>>> dtimesdiff = BinaryNode("*", d, eminusf)
>>> expression = BinaryNode("+", adivsum, dtimesdiff)
>>> expression.postorder()

```

```

a
b
c
+
/
d
e
f
-
*
+
>>>

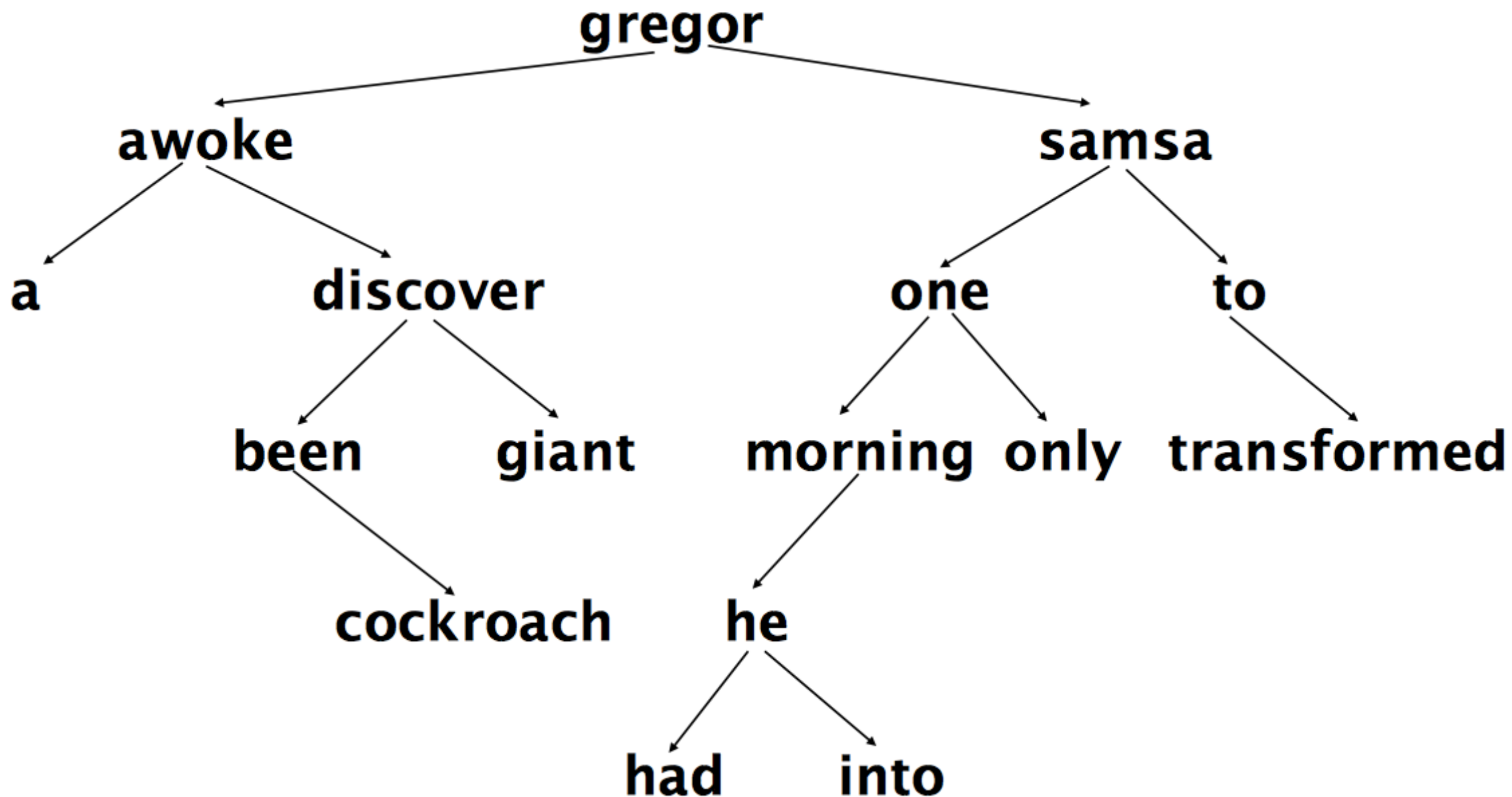
```

כתיב פולני הפוך (reverse Polish notation)
או postfix notation)
• בעזרת מחסנית אפשר לחשב את הביטוי.

עץ חיפוש בינארי

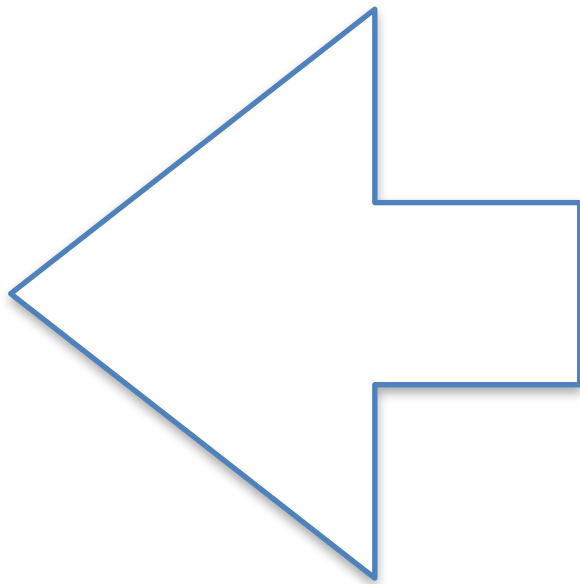
- ערכי שדה ה-data של הצמתים ניתנים להשוואה ($=$, $>$, $<$).
- אם צומת מחזיק ערך x , אז כל הצמתים בתת-העץ השמאלי שלו מחזיקים ערכים קטנים מ- x וכל הצמתים בתת-העץ הימני שלו מחזיקים ערכים גדולים מ- x .

דוגמה



מה יקרה אם נדפיס ב-inorder?

מימוש



```

>>> class BinarySearchTree:
    def __init__(self):
        self.value == None
        self.left == None
        self.right == None
    def insert(value):
        if self.value == None:
            self.value = value
        else:
            node = BinarySearchTree()
            node.value = value
            self.__insert(node)
    def __insert(self, node):
        if self.value > node.value:
            if self.left == None:
                self.left = node
            else:
                self.left.__insert(node)
        elif self.value < node.value:
            if self.right == None:
                self.right = node
            else:
                self.right.__insert(node)
    def search(self, value):
        if self.value == value:
            return True
        elif self.value > value and self.left != None:
            return self.left.search(value)
        elif self.value < value and self.right != None:
            return self.right.search(value)
        else:
            return False

```

הערות

- הביצועים תלויים בסדר ההכנסה.
- למשל, אלו מילים בסדר גרוע:

a big cat did everything –

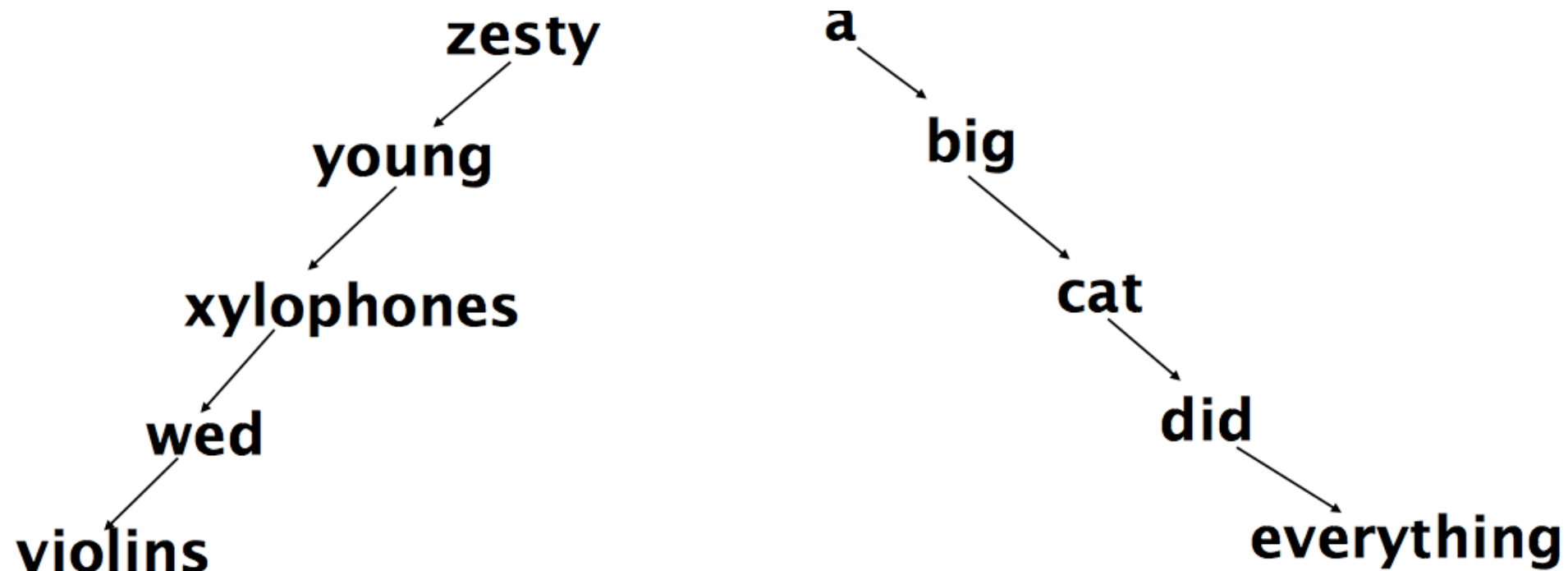
zesty young xylophones wed violins –

הערות

- הביצועים תלויים בסדר ההכנסה.
- למשל, אלו מילים בסדר גרוע:

a big cat did everything –

zesty young xylophones wed violins –



גיבוב (hashing)

מימוש מילון ללא dict

יש כמה אפשרויות לחפש פריט לפי מפתח:

- חיפוש ממצה – נעבור על כל הפריטים ונשווה את המפתח שלהם למפתח שמחפשים.
- מה גודל המילון? – עבור מנוע חיפוש, יש בערך 60 שפות שנתמכות, בעברית יש כ-120,000 מילים.
- חיפוש בינארי – נמין את המילים לפי סדר הא"ב.
- לעיתים מזומנות אנחנו צריכים משהו ממש יעיל.

כמה זה עולה לנו?

נסמן ב- n את מספר הפריטים.

- חיפוש ממצה – $O(n)$ פעולות.
- חיפוש בינארי – $O(\log n)$ פעולות.
- תקוותנו: חיפוש X – $O(1)$ פעולות.

גיבוב (hashing)

- נייצג כל מפתח ע"י מספר טבעי ייחודי.
- נשתמש במספר כאינדקס לרשימה (list).
- היתרון – התרגום של אינדקס כזה לכתובת בזיכרון המחשב בדרך כלל מאוד יעיל.
- חישוב האינדקס בהינתן המפתח חייב להיות יעיל.

```
def python_hash(key, size):  
    '''  
    :param key - hashable value  
    :param size - hash table size  
    :return Python's hash value for key, modulo size  
    '''  
    return hash(key) % size  
  
class HashEntry:  
    '''  
    An object of this class is a hash table entry storing  
    a hashable key and some data.  
    '''  
    def __init__(self, key, data):  
        self.key = key  
        self.data = data  
  
class HashTable:  
    '''  
    An object of this class is a hash table.  
    '''  
    def __init__(self, size=41, hash_func = python_hash):  
        '''  
        :param size - the size of the table  
        :param hash_func - the hash function. usage: hash_func(key, size).  
        '''  
        self._table = [None] * size  
        self._size = size  
        self._hash_func = hash_func
```

```
class HashTable:
    def __init__(self, size=41, hash_func = python_hash):

    def insert(self, key, data):
        """
        Insert an item into the hash table. If there is already an item
        with the same key, the new item overrides the old item.
        :param key - the item's key
        :param data - the item's data
        """
        self.table[self._hash_func(key, self._size)] = HashEntry(key, data)

    def delete(self, key):
        """
        Delete an item from the hash table.
        :param key - the key of the item to be deleted.
        :return True if delete successful, False if no such item in table.
        """
        index = self._hash_func(key, self._size)
        if self._table[index] and self._table[index].key == key:
            self._table[index] = None
            return True
        else:
            return False

    def search(self, key):
        """
        Return an item's data.
        :param key - the key of the item whose data should be returned.
        :return the item's data or None if no such item in table.
        """
        entry = self._table[self._hash_func(key, self._size)]
        if entry and entry.key == key:
            return entry.data
        else:
            return None
```

שימוש

```
Python 3.4.1 Shell

Python 3.4.1 (v3.4.1:c0e311e010fc, May 18 2014, 00:54:21)
[GCC 4.2.1 (Apple Inc. build 5666) (dot 3)] on darwin
Type "copyright", "credits" or "license()" for more information.
>>> ===== RESTART =====
>>>
>>> table = HashTable()
>>> table.insert('shakshuka',['tomatos','eggs','garlic','green chilli','paprika',
'white pepper','salt','vegetable oil'])
>>> table.insert('satay lembu',['sirloin','cucumber','onion','lemongrass','garlic',
',','shallot','turmeric','galangal','cumin','tamarind paste','peanuts','salt','pean
ut oil'])
>>> table.insert('til ke aloo',['potatos','cumin seeds','black mustard seeds','se
same seeds','Cayenne pepper','lemon','salt'])
>>> table.search('satay lembu')
['sirloin', 'cucumber', 'onion', 'lemongrass', 'garlic', 'shallot', 'turmeric', '
galangal', 'cumin', 'tamarind paste', 'peanuts', 'salt', 'peanut oil']
>>> python_hash('shakshuka',41)
14
>>> python_hash(14,41)
14
>>> table.search(14)
>>>
```

פונקצית גיבוב - ניסיון ראשון

- רצף הביטים של מפתח = מספר בהצגה בינארית:

- ב-unicode כל סימן מיוצג ע"י עד 16 ביטים.

- למשל: המילה 'לא' היא 98,305,488 שזה:

00000101110111000000010111010000

- במילה הארוכה ביותר בעברית יש 19 אותיות: 'וכשלאנציקלופדיותינו'.



- בסנסקריט השתמשו במילה עם 195 אותיות!

निरन्तरान्धकारिता-दिगन्तर-कन्दलदमन्द-सुधारस-बिन्दु-सान्द्रतर-घनाघन-वृन्द-सन्देहकर-स्यन्दमान-मकरन्द-बिन्दु-बन्धुरतर-माकन्द-तरु-कुल-तल्प-कल्प-मृदुल-सिकता-जाल-जटिल-मूल-तल-मरुवक-मिलदलघु-लघु-लय-कलित-रमणीय-

पानीय-शालिका-बालिका-करार-विन्द-गलन्तिका-गलदेला-लवङ्ग-पाटल-घनसार-कस्तूरिकातिसौरभ-मेदुर-लघुतर-मधुर-शीतलतर-सलिलधारा-निराकरिष्णु-तदीय-विमल-विलोचन-मयूख-रेखापसारित-पिपासायास-पथिक-लोकान्

- עוד שפות: http://en.wikipedia.org/wiki/Longest_words

- עבור עברית, נקבל list ענק 2^{16} שרובו ריק! – אין די זיכרון בעולם.

פונקצית גיבוב

- רק מעט מהרצפים של אותיות הן מילים חוקיות.
- רעיון: נעתיק את המרחב הענק של כל הרצפים האפשריים לתחום מצומצם יותר של מספרים.
- מה התכונות שאנחנו צריכים? –
 - כל מילה חוקית מקבלת מספר ייחודי לה.
 - בהינתן מילה קל לחשב את מספרה.



פונקציה רנדומית

- סימונים: n – מספר המילים, m – גודל הטבלה.
- נגדיר פונקציית ערבול h כך:
- לכל מילה w נגדיר את $h(w)$ בהתפלגות אחידה על התחום $\{0, 1, 2, \dots, m-1\}$.
- ההגרלות עבור המילים השונות בלתי תלויות לחלוטין.
- מכפיל העומס (load factor) הוא $\alpha = \frac{n}{m}$ (זו תוחלת מספר המילים ששוויכו לתא ספציפי).
- עבור $m \geq n$ הסיכוי ל-"התנגשות" $\geq \frac{1}{m} \cdot \binom{n}{2}$

פונקציה רנדומית - הערות

- הסיכוי ל-"התנגשות" $\binom{n}{2} \cdot \frac{1}{m} \geq$

- אם $m = n^2$ הסיכוי להתנגשות $\geq \frac{1}{2}$

- אם $m \approx \frac{2}{\pi} n^2$ אז הסיכוי להתנגשות $\leq \frac{1}{2}$

- אם $m = n$ אז תוחלת העומס המקסימלי על תא היא $\frac{\ln n}{\ln \ln n} \approx$
הסיכוי של תא להיות ריק הוא $e^{-1} \approx$

- קשה לייצג ולחשב פונקציה רנדומית.

- לכן, משתמשים בפונקציות פסאודו-רנדומיות.

פונקציה רנדומית - הערות

- הסיכוי ל-"התנגשות" $\binom{n}{2} \cdot \frac{1}{m} \geq$

- אם $m = n^2$ הסיכוי להתנגשות $\geq \frac{1}{2}$

- אם $m \approx \frac{2}{\pi} n^2$ אז הסיכוי להתנגשות $\leq \frac{1}{2}$

$$\frac{\ln n}{\ln \ln n}$$

- אם $m = n$ אז תוחלת העומס המקסימלי על תא היא $\approx$ הסיכוי של תא להיות ריק הוא $\approx e^{-1}$.

- קשה לייצג ולחשב פונקציה רנדומית.

- לכן, משתמשים בפונקציות פסאודו-רנדומיות.

פונקציה רנדומית - הערות

- הסיכוי ל-"התנגשות" $\binom{n}{2} \cdot \frac{1}{m} \geq$



- אם $m = n^2$ הסיכוי להתנגשות $\geq \frac{1}{2}$

- אם $m \approx \frac{2}{\pi} n^2$ אז הסיכוי להתנגשות $\leq \frac{1}{2}$

$$\frac{\ln n}{\ln \ln n}$$

- אם $m = n$ אז תוחלת העומס המקסימלי על תא היא $\approx e^{-1}$.
הסיכוי של תא להיות ריק הוא $\approx e^{-1}$.

- קשה לייצג ולחשב פונקציה רנדומית.

- לכן, משתמשים בפונקציות פסאודו-רנדומיות.

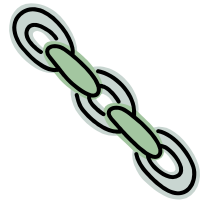
פונקצית הגיבוב של פייתון

- פונקצית הספרייה hash פועלת על כל אובייקט שיש לו מתודה `.__hash__`.
- לא ממש רנדומית! אבל נמנעת מהתנגשויות היטב.
- הטווח גדול (וכלל מספרים שליליים).
- דוגמה (קטע מהמחלקה string):

```
class string:
    def __hash__(self):
        if not self:
            return 0 #empty
        value = ord(self[0]) << 7
        for char in self:
            value = c_mul(1000003, value) ^ ord(char)
        value = value ^ len(self)
        if value == -1:
            value = -2
        return value
```

טיפול בהתנגשויות

- אם רוצים טבלה קטנה ומשתמשים פונקצית גיבוב שלא תלוייה במילים החוקיות, יש סיכוי גבוה להתנגשויות.
- איך מטפלים בהתנגשויות? – יש שתי גישות:
 - שרשור (chaining).
 - מיעון פתוח (open addressing).



שרשור

- כל תא בטבלה הוא דלי שמחזיק רשימה של כל הפריטים שמופואליו.
- כאשר מחפשים פריט עם מפתח k , אז $h(k)$ מפנה לדלי הנכון ושם צריך לחפש את הפריט.
- בפייתון אפשר להחזיק את התוכן של כל דלי כ-`list` (אפשר גם כרשימה מקושרת).
- על מנת למצוא פריט בדלי, צריך לעבור על תאי ה-`list` לפי הסדר.
- העומס המירבי על תא בטבלה קובע את סיבוכיות החיפוש.


```
class HashTable:
    def insert(self, key, data):

    def delete(self, key):
        """
        Delete an item from the hash table.
        :param key - the key of the item to be deleted.
        :return True if delete successful, False if no such item in table.
        """
        index = self._hash_func(key, self._size)
        for i in range(len(self._table[index])):
            if self._table[index][i].key == key:
                self._table[index].pop(i)
                return True
        return False

    def search(self, key):
        """
        Return an item's data.
        :param key - the key of the item whose data should be returned.
        :return the item's data or None if no such item in table.
        """
        index = self._hash_func(key, self._size)
        for entry in self._table[index]:
            if entry.key == key:
                return entry.data
        return None
```

מיעון פתוח

- כל תא בטבלה מחזיק לכל היותר פריט אחד.
- אם רוצים להכניס פריט לתא תפוש, מזיזים את הפריט הישן (או החדש) לתא אחר על פי נוסחה ידועה.
- הזזה כזו יכולה לגרום לשרשרת של הזזות.
- על מנת למצוא פריט צריך לעבור לפי הסדר על התאים בהם הוא עשוי להימצא.
- דוגמה: בסריקה ליניארית (linear probing) עוברים על התאים בדילוג קבוע (למשל - 1) החל מהתא שרוצים עד שמוצאים תא פנוי.

```

class HashTable:
    '''
    An object of this class is a hash table. The worst case time complexity
    for insert, delete, and search is Omega(self._size), but it is often much
    better. (For delete and search the worst case always happens if key is
    not in the table.)
    '''
    def __init__(self, size=41, hash_func = python_hash):
        '''
        The table is implemented using linear probing with step size 1.
        :param size - the size of the table
        :param hash_func - the hash function. usage: hash_func(key, size).
        '''
        self._table = [None] * size
        self._size = size
        self._hash_func = hash_func

    def insert(self, key, data):
        '''
        Insert an item into the hash table.
        If the table is full, the new item is not added.
        :param key - the item's key
        :param data - the item's data
        :return True if the item was added, False otherwise.
        '''
        index = self._hash_func(key, self._size)
        for i in range(self._size):
            place = (index+i) % self._size
            if not self._table[place]:
                self._table[place] = HashEntry(key, data)
                return True
        return False

```

```
class HashTable:
    def insert(self, key, data):

    def delete(self, key):
        """
        Delete an item from the hash table.
        :param key - the key of the item to be deleted.
        :return True if delete successful, False if no such item in table.
        """
        index = self._hash_func(key, self._size)
        for i in range(self._size):
            place = (index+i) % self._size
            if self._table[place].key == key:
                self._table[place] = None
                return True
        return False

    def search(self, key):
        """
        Return an item's data.
        :param key - the key of the item whose data should be returned.
        :return the item's data or None if no such item in table.
        """
        index = self._hash_func(key, self._size)
        for i in range(self._size):
            place = (index+i) % self._size
            if self._table[place].key == key:
                return self._table[place].data
        return None
```